

LANGUAGE PL/SQL

Langage PL/SQL



PLAN

- ☐ Introduction
- ☐ Avantages de PL/SQL
- ☐ Les Principales caractéristique du Pl / SQL
- ☐ Les Variables et les constantes
- ☐ Les instructions de base
- ☐ Interaction avec la base de données à partir de PL/SQL
- ☐ Les curseurs
- ☐ Gestion de exceptions
- ☐ Les procédures, exceptions
- ☐ Les triggers

Introduction



- ❑ PL/SQL (Procedural Language / SQL), l'extension procédural proposée par Oracle pour SQL,
- ❑ Il permet de combiner des requêtes SQL (SELECT, INSERT, UPDATE et DELETE) et des instructions procédurales (boucles, conditions...),
- ❑ Créer des traitements complexes destinés à être stockés sur le serveur de base de données
- ❑ Comme on le sait, les structures de contrôle habituelles d'un langage (IF, WHILE..) ne font pas partie intégrante de la norme SQL. Oracle les prend en compte dans PL/SQL.

Introduction

Id_Produit	Libellé	Marque	Prix	ID_Fournisseur
P1	HP600	HP	1000	1
P2	Epson30	Epson	2300	1
P3	IBM 7380	IBM	4600	2
P4	Lexmark310	Lexmark	2000	3

❑ Update produit set prix=2500 where id_fournisseur=1

CODE PL/SQL

Procedure :

IF ... THEN

ELSEIF

ELSE

END IF

❑ Pour le fournisseur N°1

Si Moyenne(prix)>2000

Augmenter le prix de 5%

Sinon

Augmenter le prix de 3%

Avantage du langage PL/SQL

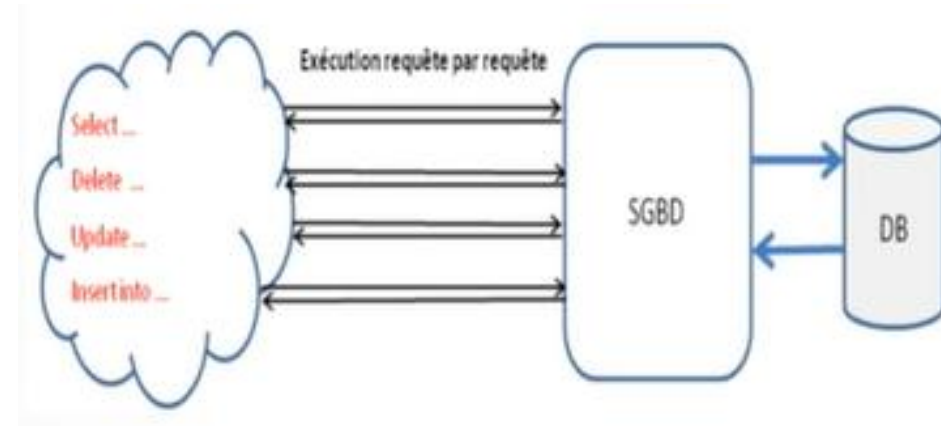


Ce langage propose des performances pour le traitement des transactions et offre les avantages suivants :

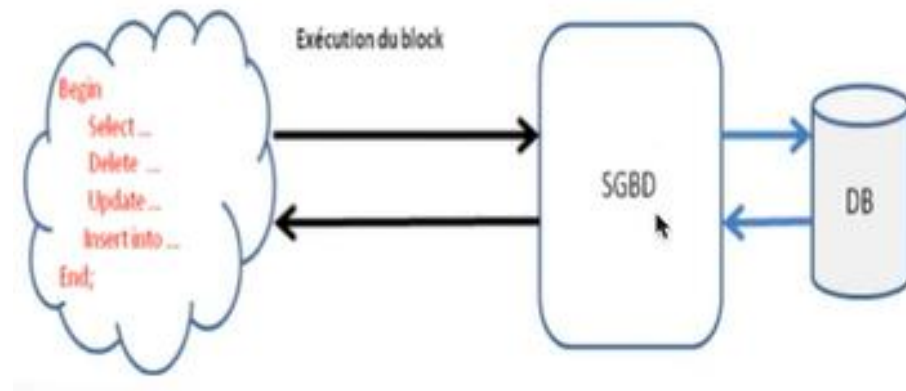
- ❑ Intégration complète du SQL
- ❑ Prise en charge de la programmation orientée objet (surcharge),
- ❑ Parfaite intégration avec Oracle et Java, en fait **PL/SQL** est un langage propriétaire de Oracle,
 - On peut lancer des sous-programme PL/SQL à partir de **Java** et de même, on peut appeler des procédures Java à partir d'un block PL/SQL

Avantage du langage PL/SQL

Dans un environnement client-serveur, chaque instruction SQL donne lieu à l'envoi d'un message du client vers le serveur suivi de la réponse du serveur vers le client



Un bloc PL/SQL donne lieu à un seul échange sur le réseau entre le client et le serveur. Les résultats intermédiaires sont coté serveur et seul le résultat final est retourné au client.



Langage PL/SQL



Un programme ou une procédure PL/SQL est un ensemble d'un ou plusieurs blocs. Chaque bloc comporte trois sections :

- Section déclaration
- Section Corps du bloc
- Section traitement des erreurs (exceptions)

Structure de bloc PL/SQL

[DECLARE] (facultatif)

Déclarer les objets PL/SQL à utiliser dans ce bloc

BEGIN (obligatoire)

Définir les instructions exécutables

[EXCEPTION] (facultatif)

Définir les actions à entreprendre en cas d'erreur ou d'exception

END; (obligatoire)

Sub Block

DECLARE

BEGIN

EXCEPTION

END;

Structure de bloc PL/SQL

Les types des blocs

Anonymous

```
[DECLARE]

BEGIN
    --statements

[EXCEPTION]

END;
```

Procedure

```
PROCEDURE name
IS

BEGIN
    --statements

[EXCEPTION]

END;
```

Function

```
FUNCTION name
RETURN datatype
IS

BEGIN
    --statements

[EXCEPTION]

END;
```

Outils de programmation PL/SQL

```
C:\> sqlplus
```

```
DECLARE
```

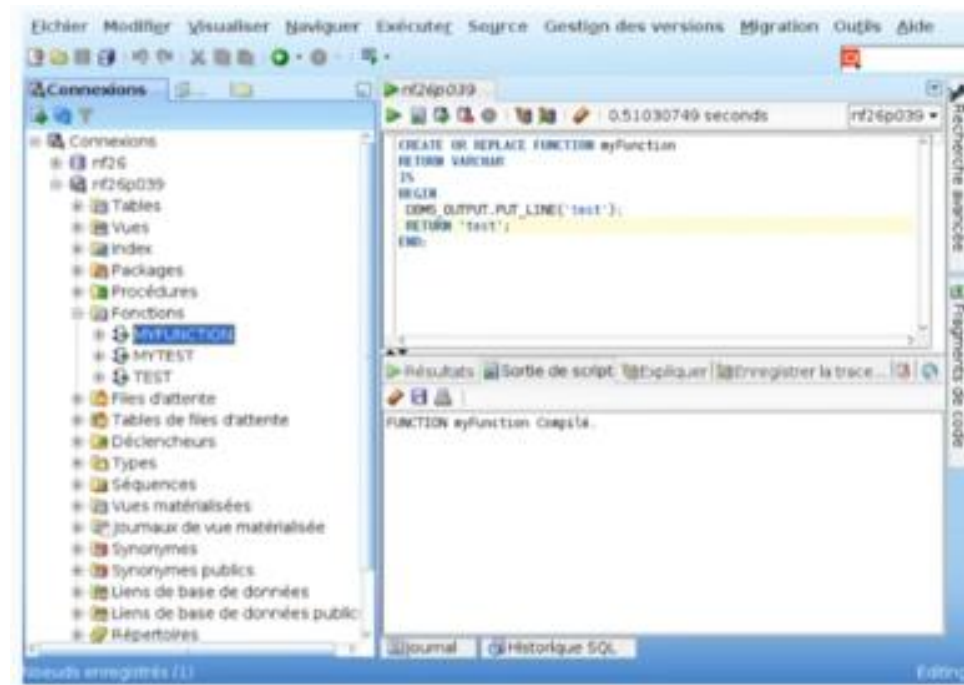
```
    v_telephone NUMBER(10);  
    v_disponible BOOLEAN;  
    v_DateDeNaissance Date;  
    c_TVA CONSTANT NUMBER :=0.02;  
    v_TTC NUMBER ;  
    v_HT NUMBER ;
```

```
BEGIN
```

```
    v_disponible :=TRUE ;  
    v_telephone :=0666666666;  
    v_TTC := v_HT + c_TVA ;
```

```
END;
```

```
/
```



Outil: SQLPLUS

Outil: SQL Developer

Mon premier programme PL/SQL

```
SET SERVEROUTPUT ON;
```

```
BEGIN
```

```
  DBMS_OUTPUT.put_line('Bonjour tout le monde');
```

```
END;
```

```
/
```



PL/SQL

Déclaration des variables

Déclaration des variables

- Les types habituels correspondants au SQL2 ou Oracle : **integer, varchar, ...**
- Les types composites adaptés à la récupération des colonnes et des lignes des SQL ; **%TYPE, %ROWTYPE**

Règles :

- Adopter les conventions de dénomination des objets.
- Initialiser les constantes et les variables déclarées **NOT NULL**.
- Initialiser les identifiants en utilisant l'opérateur d'affectation (**:=**) ou le mot réservé **DEFAULT**
- Déclarer au plus un identifiant par ligne.

Déclaration des variables

Syntaxe:

identifier [CONSTANT] *datatype* [NOT NULL]
[:= | DEFAULT *exp*];

Exemple :

DECLARE

```
v_DateNaissance    DATE;
v_NDept            Number(2) NOT NULL :=10;
v_Ville            VARCHAR2(13) := 'Aix';
v_CodeP            CONSTANT NUMBER := 13100 ;
v_Valide           BOOLEAN NOT NULL := TRUE;
v_greet            VARCHAR2(20) DEFAULT 'have a Goode day';
```

Déclaration des variables

Les types d'oracle

TYPE	VALEURS
BINARY-INTEGER	entiers allant de -2^{31} à 2^{31}
POSITIVE / NATURAL	entiers positifs allant jusqu'à $2^{31} - 1$
BUMBER	Numérique entre -2^{418} à 2^{418}
INTEGER	Entier stocké en binaire entre -2^{126} à 2^{126}
CHAR (n)	Chaine fixe de 1 32767 caractères (différent pour une colonne de table)
VARCHAR2 (n)	Chaine variable de 1 à 32767 caractères
LONG	Idem VARCHAR2 (maximum 2 gigaoctet)
RAW	Permet de stocker des types de données binaire relativement faibles (≤ 32767 octet) idem VARCHAR2
LON RAW	Idem LONG mais avec du binaire
CLOB	Données d'objet de grande taille de type caractère. Objets de type long stockés en binaire (maximum 4 giga octets)

Déclaration des variables avec l'attribut %Type

Exemple :

```
...  
    v_Nom          client.nom%TYPE;  
    v_age          NUMBER(7,2);  
    v_min_age      v_age%TYPE := 10;  
...
```

RQ : NOT NULL ne s'applique pas aux variables déclarées avec %TYPE

Créer un enregistrement PL/SQL

Déclare des variables pour le stockage du nom, du poste et du salaire d'un nouvel employé

Exemple :

```
...  
TYPE emp_record_type IS RECORD  
  ( nom      VARCHAR2(25) ,  
    fonction  VARCHAR2(10) ,  
    salaire   NUMBER(8,2)  
  );  
emp_record   emp_record_type;
```

RQ : - NOT NULL peut être appliqué au champ
- champ peut être déclaré avec %TYPE

Entrée-sortie PL/SQL

Il est évidemment possible d'utiliser des fonctions prédéfinies d'entrée/sortie. Ces fonctions servent essentiellement durant la phase de test de procédure PL/SQL

Pour entrer une valeur : **ACCEPT** msg **PROMPT**

Pour afficher une valeur : **DBMS_OUTPUT.put_line**('Test : ' || message);

```
ACCEPT msg PROMPT 'Entrer votre message'  
DECLARE  
message varchar2(100):='&msg';  
BEGIN  
  DBMS_OUTPUT.put_line('TEST: ' || message)  
END;
```



Exemples : Variable

The screenshot displays the SQL Developer interface with the following components:

- Menu Bar:** Fichier, Modifier, Affichage, Naviguer, Exécuter, Source, Equipe, Outils, Fenêtre, Aide.
- Toolbox:** Contains icons for file operations, database connections, and execution.
- Left Panel:**
 - Connexions:** Lists Oracle connections including 'mydb' and 'ORCL'.
 - Rapports:** Lists various report types such as 'Rapports Data Modeler', 'Rapports de dictionnaire de données', 'Rapports définis par l'utilisateur', 'Rapports de vue analytique', 'Rapports OLAP', and 'Rapports TimesTen'.
- Main Window:**
 - Page de bienvenue** and **variables.sql** tabs are open.
 - Feuille de calcul SQL** and **Historique** tabs are visible.
 - Query Builder:** Shows the following PL/SQL script:

```
DECLARE
var_nom VARCHAR2(20);
var_date DATE;
Var_age NUMBER;
BEGIN
var_nom := 'HADID';
var_age := 23;
var_date := '01/04/2020';
DBMS_OUTPUT.PUT_LINE('Le nom : ' || var_nom);
DBMS_OUTPUT.put_LINE('La date : ' || var_date);
DBMS_OUTPUT.PUT_LINE('L'age : ' || var_age);
END;
```
 - Sortie de script:** Displays the execution results:

```
Le nom :HADID
La date :01/04/20
L'age :23

Procédure PL/SQL terminée.
```

Exemples : Variable

The screenshot displays the Oracle SQL Developer environment. The top window, titled 'Feuille de calcul SQL', contains a PL/SQL script. The script declares three variables: `var_nom` of type `VARCHAR2(20)` with value `'TOTO'`, `var_date` of type `DATE` with value `'02/03/2019'`, and `Var_age` of type `NUMBER` with value `30`. It then uses `DBMS_OUTPUT.PUT_LINE` to print the values of these variables. The bottom window, titled 'Sortie de script', shows the output of the script execution, which matches the values declared in the script. The status bar indicates the task was completed in 0.093 seconds.

```
set SERVEROUTPUT ON
DECLARE
  var_nom VARCHAR2(20):='TOTO';
  var_date DATE:='02/03/2019';
  Var_age NUMBER :=30;
BEGIN
  DBMS_OUTPUT.PUT_LINE('Le nom : ' || var_nom);
  DBMS_OUTPUT.put_LINE('La date : ' || var_date);
  DBMS_OUTPUT.PUT_LINE('L'age : ' || var_age);
END;
```

Le nom :TOTO
La date :02/03/19
L'age :30

Procédure PL/SQL terminée.

Exemples : Variable

The screenshot displays the Oracle SQL Developer environment. The top window, titled 'Feuille de calcul SQL', contains a PL/SQL script. The script declares three variables: `var_nom` of type `VARCHAR2(20)` with value `'TOTO'`, `var_date` of type `DATE` with value `'02/03/2019'`, and `Var_age` of type `NUMBER` with value `30`. It then uses `DBMS_OUTPUT.PUT_LINE` to print the values of these variables. The bottom window, titled 'Sortie de script', shows the output of the script execution, which matches the values declared in the script. The status bar indicates the task was completed in 0.093 seconds.

```
set SERVEROUTPUT ON
DECLARE
  var_nom VARCHAR2(20):='TOTO';
  var_date DATE:='02/03/2019';
  Var_age NUMBER :=30;
BEGIN
  DBMS_OUTPUT.PUT_LINE('Le nom : ' || var_nom);
  DBMS_OUTPUT.put_LINE('La date : ' || var_date);
  DBMS_OUTPUT.PUT_LINE('L'age : ' || var_age);
END;
```

Le nom :TOTO
La date :02/03/19
L'age :30

Procédure PL/SQL terminée.

Exemples : Variable %TYPE

The screenshot displays the Oracle SQL Developer interface. On the left, the 'Connexions' pane shows a tree view of the 'mydb' database, including tables like 'EMPLOYEES' with columns 'ID', 'NAME', 'DEPARTMENT', and 'SALARY'. The main window is titled 'Feuille de calcul SQL' and contains a SQL script. The script declares three variables: 'var_nom' of type 'employees.name%TYPE', 'var_date' of type 'DATE' with a value of '02/03/2025', and 'Var_age' of type 'NUMBER' with a value of 30. The script then assigns 'TITI' to 'var_nom' and uses 'DBMS_OUTPUT.PUT_LINE' to display the values. The bottom pane, 'Sortie de script', shows the execution results: 'Le nom :TITI', 'La date :02/03/25', and 'L'age :30'. The status bar indicates 'Procédure PL/SQL terminée.' and 'Tâche terminée en 0,046 secondes'.

```
set SERVEROUTPUT ON
DECLARE
  var_nom employees.name%TYPE;
  var_date DATE := '02/03/2025';
  Var_age NUMBER := 30;
BEGIN

  var_nom:= 'TITI';

  DBMS_OUTPUT.PUT_LINE('Le nom : ' || var_nom);
  DBMS_OUTPUT.put_LINE('La date : ' || var_date);
  DBMS_OUTPUT.PUT_LINE('L'age : ' || var_age);
END;
```

Sortie de script x

Tâche terminée en 0,046 secondes

Le nom :TITI
La date :02/03/25
L'age :30

Procédure PL/SQL terminée.

Exemples : Variable (enregistrement)

The screenshot displays the Oracle SQL Developer environment. On the left, the 'Connexions' pane shows a connection to 'mydb' with a tree view of database objects including 'EMPLOYEES' (with columns ID, NAME, DEPARTMENT, SALARY), 'Vues', 'Index', 'Packages', 'Procédures', 'Fonctions', and 'Opérateurs'. Below this is the 'Rapports' pane with various report categories.

The main window is titled 'Feuille de calcul SQL' and shows a script in the 'Feuille de calcul' tab. The script is as follows:

```
set SERVEROUTPUT ON
DECLARE
TYPE enreg_personne is RECORD
(
  var_nom employees.name%TYPE,
  var_date DATE := '02/03/2025',
  Var_age NUMBER := 30
);

personne enreg_personne;
BEGIN

personne.var_nom:= 'TITI';

DBMS_OUTPUT.PUT_LINE('Le nom : ' || personne.var_nom);
DBMS_OUTPUT.put_LINE('La date : ' || personne.var_date);
DBMS_OUTPUT.PUT_LINE('L'age : ' || personne.var_age);
END;
```

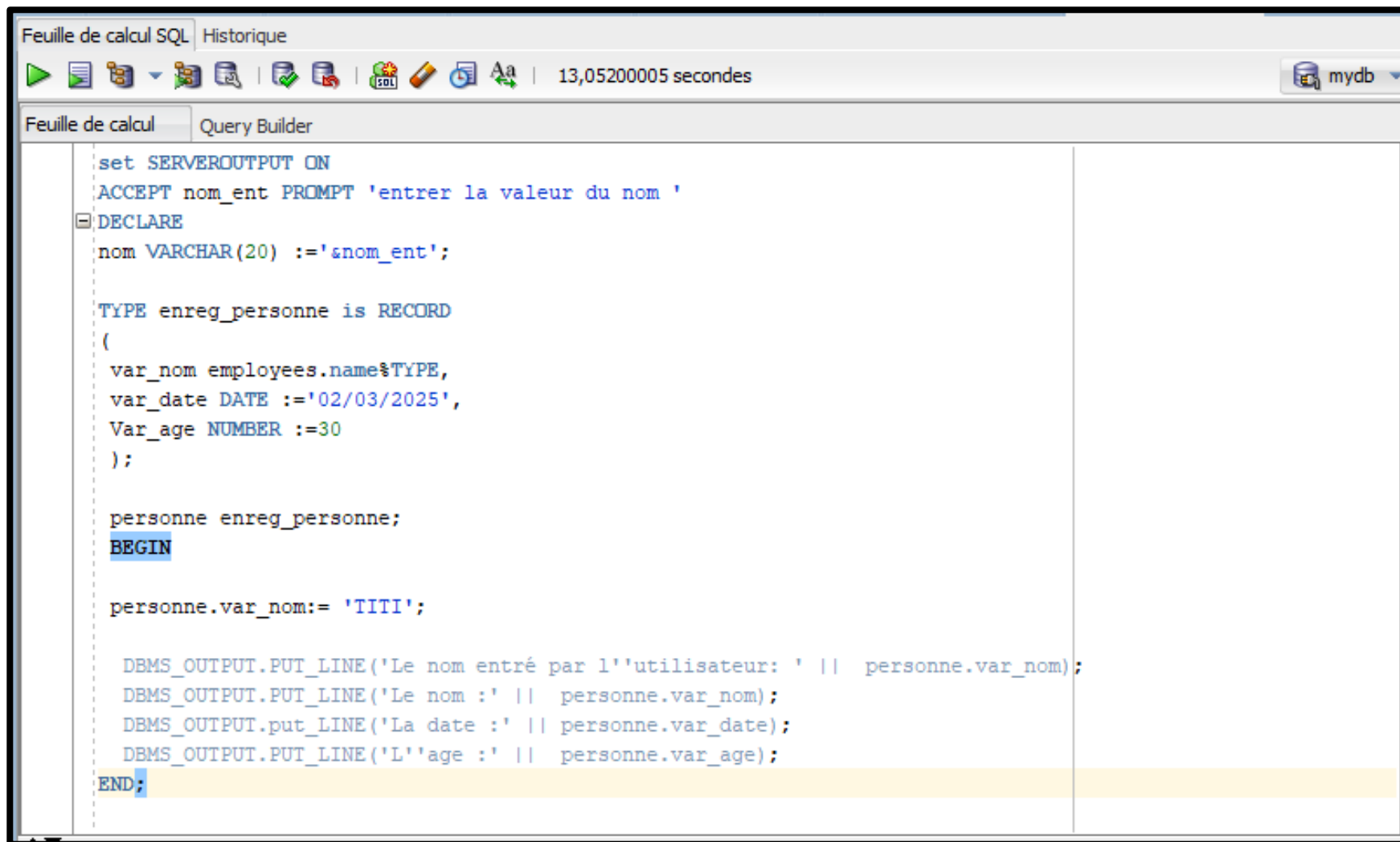
The 'Sortie de script' pane at the bottom shows the execution results:

```
Le nom :TITI
La date :02/03/25
L'age :30

Procédure PL/SQL terminée.
```

The status bar indicates the task was completed in 0,057 seconds.

Exemples : Variable (Entrer)



The screenshot shows a SQL IDE window titled "Feuille de calcul SQL" with a "Historique" tab. The main editor area is titled "Feuille de calcul" and "Query Builder". The script contains the following PL/SQL code:

```
set SERVEROUTPUT ON
ACCEPT nom_ent PROMPT 'entrer la valeur du nom '
DECLARE
nom VARCHAR(20) := '&nom_ent';

TYPE enreg_personne is RECORD
(
  var_nom employees.name%TYPE,
  var_date DATE := '02/03/2025',
  Var_age NUMBER := 30
);

personne enreg_personne;
BEGIN

personne.var_nom:= 'TITI';

DBMS_OUTPUT.PUT_LINE('Le nom entré par l'utilisateur: ' || personne.var_nom);
DBMS_OUTPUT.PUT_LINE('Le nom : ' || personne.var_nom);
DBMS_OUTPUT.put_LINE('La date : ' || personne.var_date);
DBMS_OUTPUT.PUT_LINE('L'age : ' || personne.var_age);
END;
```

The IDE interface includes a toolbar with icons for execution, saving, and other database operations. The status bar at the bottom indicates a execution time of 13,05200005 secondes. The database connection is set to "mydb".



PL/SQL

Les instructions de base

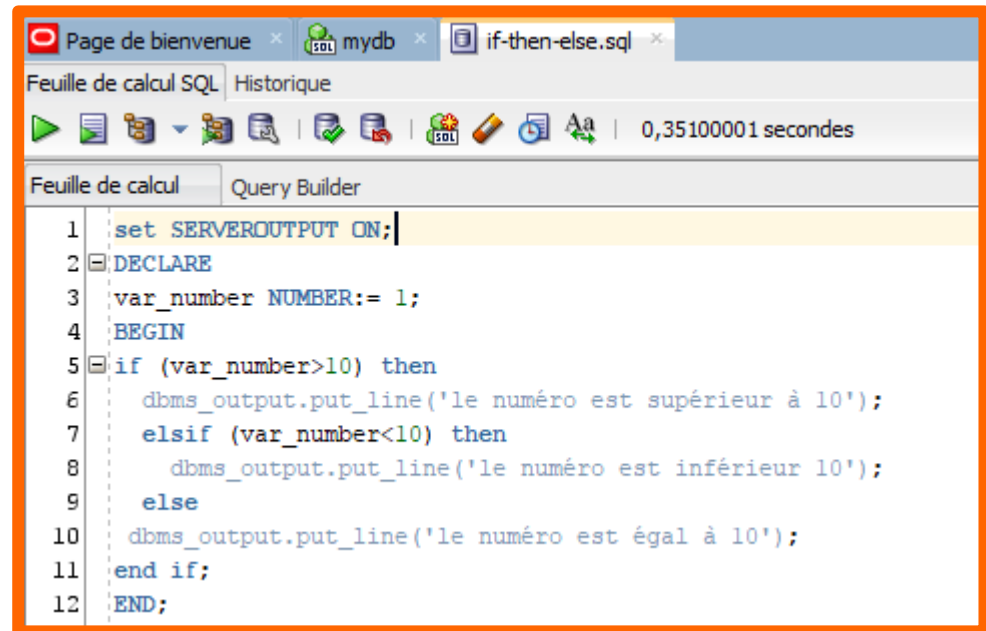
Instructions de base

IF-THEN-ELSE

L'instruction IF-ELSE est une des premières structures conditionnelles que l'on apprend en programmation. Elle est composée d'une condition. Si cette condition est vraie, alors les instructions après le IF sont exécutées. Si la condition est fausse, ce sont les instructions sous le ELSE qui sont exécutées.

Exemple :

```
. . .  
IF v_start > 100 THEN  
    v_start := 2 * v_start;  
  
ELSIF v_start >= 50 THEN  
    v_start := 0.5 * v_start;  
ELSE  
    v_start := 0.1 * v_start;  
END IF;  
. . .
```



The screenshot shows a SQL IDE window with a tab titled 'if-then-else.sql'. The interface includes a 'Feuille de calcul SQL' (SQL Worksheet) and a 'Query Builder' tab. The SQL code is as follows:

```
1 set SERVEROUTPUT ON;  
2 DECLARE  
3 var_number NUMBER:= 1;  
4 BEGIN  
5 if (var_number>10) then  
6     dbms_output.put_line('le numéro est supérieur à 10');  
7     elsif (var_number<10) then  
8         dbms_output.put_line('le numéro est inférieur 10');  
9     else  
10        dbms_output.put_line('le numéro est égal à 10');  
11    end if;  
12 END;
```

The execution time is displayed as 0,35100001 secondes.

Instructions de base

Instruction Case

Dans le langage SQL, la commande "CASE ...WHEN.." permet d'utiliser des conditions de type "si / sinon" similaire à un langage de programmation pour retourner un résultat disponible entre plusieurs possibilités

Exemple :

```
DECLARE
Mention char(1);
BEGIN
CASE
    WHEN Mention ='T' THEN
        DBMS_OUTPUT.PUT_LINE('Mention Très bien');
    WHEN Mention ='B' THEN
        DBMS_OUTPUT.PUT_LINE('Mention bien');
    ELSE
        DBMS_OUTPUT.PUT_LINE('Mention Passable');
    END CASE;
END;
```

Instructions de base

Instruction Case

Dans le langage SQL, la commande "CASE ...WHEN.." permet d'utiliser des conditions de type "si / sinon" similaire à un langage de programmation pour retourner un résultat disponible entre plusieurs possibilités

Exemple :

A screenshot of a SQL Query Builder window. The window has two tabs: 'Feuille de calcul' and 'Query Builder'. The 'Query Builder' tab is active, showing a SQL script. The script is as follows:

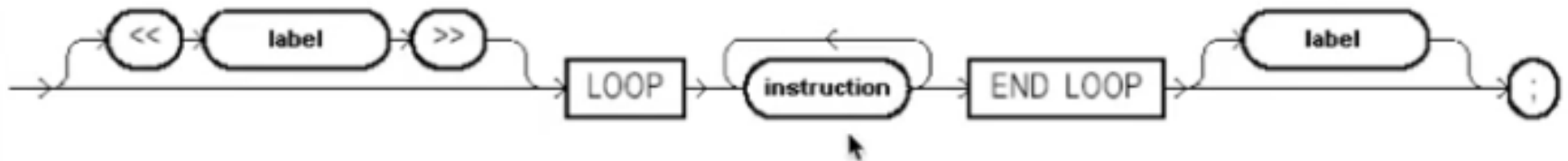
```
1 set SERVEROUTPUT ON;
2 DECLARE
3   var_note NUMBER:= 11;
4 BEGIN
5 CASE
6   when var_note<10 then
7     dbms_output.put_line('la mention: mal');
8
9   when var_note>=10 and var_note<12 then
10    dbms_output.put_line('la mention: passable');
11
12   when var_note>=12 and var_note<14 then
13    dbms_output.put_line('la mention: Assez bien');
14
15   when var_note>=14 and var_note<16 then
16    dbms_output.put_line('la mention: Bien');
17
18   else
19    dbms_output.put_line('la mantion : Très Bien');
20 end CASE;
21 END;
```

The script is numbered from 1 to 21. The 'END;' statement at line 21 is highlighted in yellow. The window title bar shows 'Feuille de calcul' and 'Query Builder'.

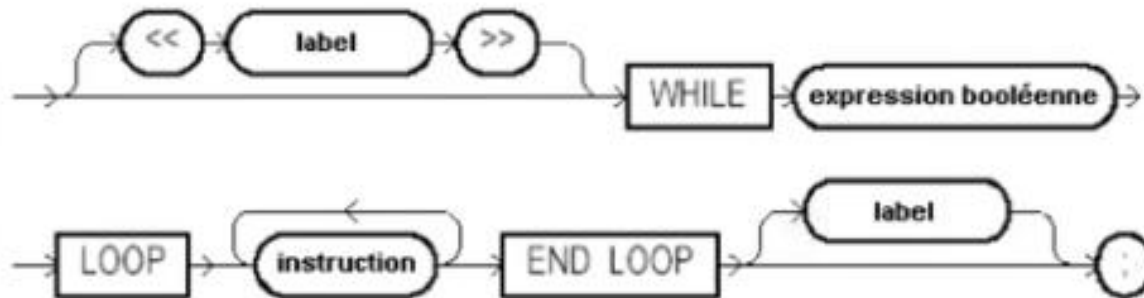
Instructions de base

Les boucles

Boucle LOOP



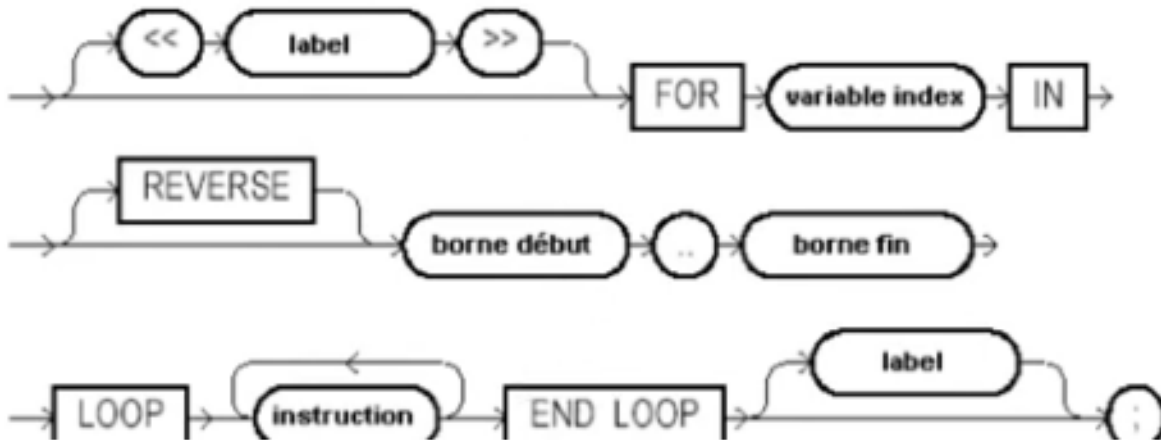
Boucle WHILE



Instructions de base

Les boucles

Boucle FOR



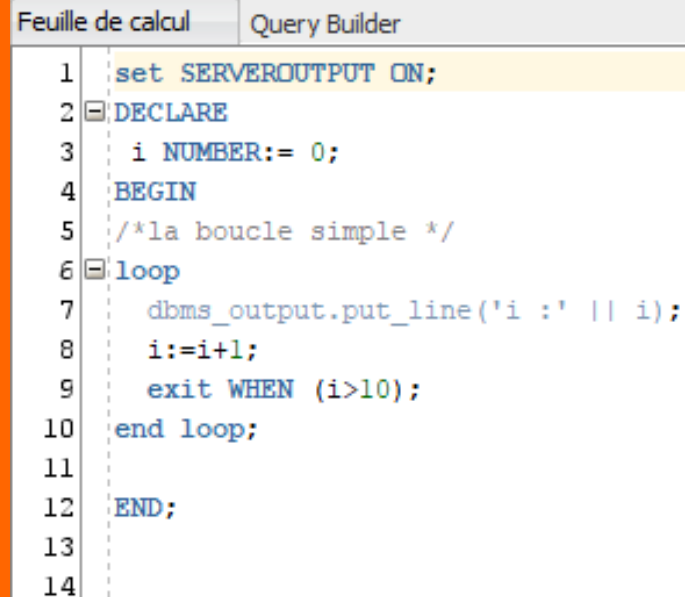
Instructions de base

Boucle simple

Exemple :

```
DECLARE
    Compte Number;
BEGIN
    LOOP
        DBMS_OUTPUT.PUT_LINE(Compte);

        compte := compte + 1;
        EXIT WHEN compte > 10;
    END LOOP;
END;
```



The screenshot shows a SQL query editor window with two tabs: 'Feuille de calcul' and 'Query Builder'. The 'Query Builder' tab is active, displaying a SQL script with line numbers 1 through 14. The script is a PL/SQL block that sets server output on, declares a variable 'i' as a number and initializes it to 0, then enters a loop. Inside the loop, it prints the value of 'i', increments it by 1, and exits when 'i' is greater than 10. The loop ends with 'end loop;' and the entire block ends with 'END;'.

```
1 set SERVEROUTPUT ON;
2 DECLARE
3     i NUMBER:= 0;
4 BEGIN
5     /*la boucle simple */
6 loop
7     dbms_output.put_line('i :' || i);
8     i:=i+1;
9     exit WHEN (i>10);
10 end loop;
11
12 END;
13
14
```

Instructions de base

Boucle FOR

Exemple :

```
DECLARE

BEGIN
  FOR i IN 1..10 LOOP
    DBMS_OUTPUT.PUT_LINE(i);
  END LOOP;
END;
```

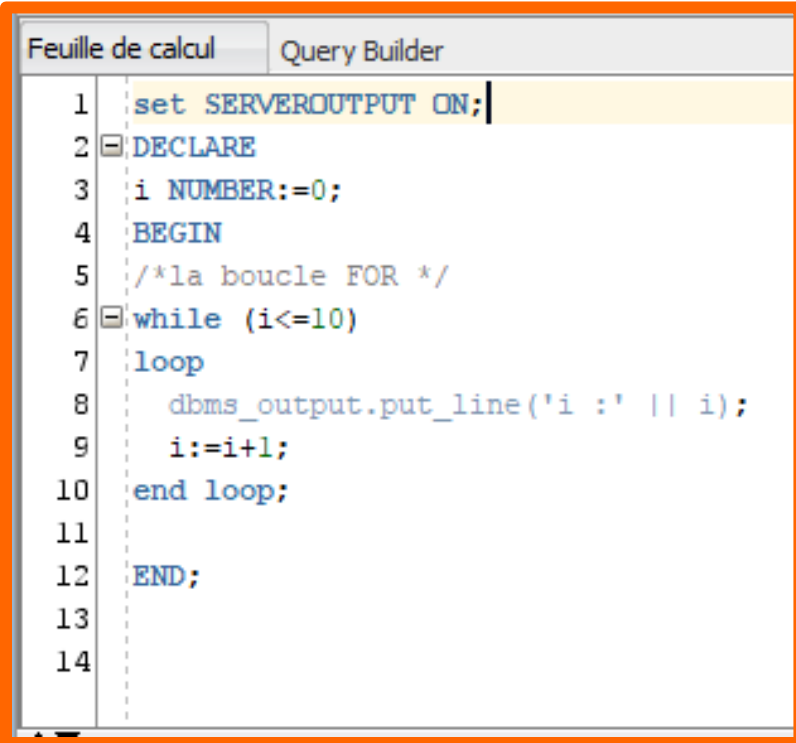
	Feuille de calcul	Query Builder
1		set SERVEROUTPUT ON;
2	☐	DECLARE
3		
4		BEGIN
5		/*la boucle FOR */
6	☐	for i in 1..10
7		loop
8		dbms_output.put_line('i :' i);
9		end loop;
10		
11		END;
12		

Instructions de base

Boucle WHILE

Exemple :

```
DECLARE
i      INTEGER;
BEGIN
  While i < 10 LOOP
    DBMS_OUTPUT.PUT_LINE(i);
    i:=i+1;
  END LOOP;
END;
```



The screenshot shows a SQL Query Builder window with two tabs: "Feuille de calcul" and "Query Builder". The "Query Builder" tab is active, displaying a PL/SQL script. The script is as follows:

```
1  set SERVEROUTPUT ON;
2  DECLARE
3  i NUMBER:=0;
4  BEGIN
5  /*la boucle FOR */
6  while (i<=10)
7  loop
8      dbms_output.put_line('i :' || i);
9      i:=i+1;
10 end loop;
11
12 END;
```



PL/SQL

Interaction avec la base de données
Les collections

Interaction avec la base de base

La clause INTO en PL/SQL

La clause **INTO** permet de passer des valeurs d'une table dans des variables

Exemple :

```
DECLARE
    v_id NUMBER(4);
    v_ville varchar(20);
BEGIN
    SELECT id_Emp, ville_Emp
    INTO v_id, v_ville
    FROM Employe
    WHERE nom_Emp= 'Ahmadi';
    ...
END;
```

Interaction avec la base de base

Mettre à jour des données

Augmenter le salaire de tous les employés du service **commercial** dans la table EMPLOYE

Exemple :

```
DECLARE
    v_salaire    employe.salaire%TYPE := 2000;
BEGIN
    UPDATE employe
    SET    salaire = salaire + v_salaire
    WHERE  service= 'commercial';
END;
```

RQ: En l'absence de modification de ligne, aucune erreur ne se produit

Interaction avec la base de base

Supprimer les données

Supprimer les lignes appartenant **service achat** à partir de la table EMPLOYE

Exemple :

```
DECLARE
    v_service    employe.service%TYPE := 'achat';
BEGIN
    DELETE FROM employees
    WHERE service = v_service;
END;
```

Instructions COMMIT et ROLLBACK



- ❑ Initialiser une transaction avec la première instruction **LMD** suivant **COMMIT** ou **ROLLBACK**
- ❑ Utiliser les instructions **SQL COMMIT** ou **ROLLBACK** pour mettre fin explicitement à une transaction

Déclarer des variables avec l'attribut %ROWTYPE

Déclarer une variable permettant de stocker les mêmes informations relatives à un employé telles qu'elles figurent dans la table EMPLOYEES.

Exemple :

```
DECLARE
    Emp_record  Employees%ROWTYPE;
    ...
BEGIN
    SELECT * INTO Emp_record
    FROM   Employees
    WHERE  ...
```

Déclarer des variables avec l'attribut %ROWTYPE

Exemple :

The screenshot displays the Oracle SQL Developer interface. On the left, the 'Connexions' pane shows a connection to 'mydb' with a tree view of the database schema, including tables like EMPLOYEES and views. The main window is titled 'Feuille de calcul SQL' and contains a script named 'RowType1.sql'. The script declares a variable 'v_employee' of type 'EMPLOYEES%ROWTYPE', selects data from the 'employees' table where 'id=2', and outputs the results using 'dbms_output.put_line'. The 'Sortie de script' pane at the bottom shows the execution results, including the employee's ID, name, department, and salary, followed by the message 'Procédure PL/SQL terminée.'

```
DECLARE
v_employee EMPLOYEES%ROWTYPE;

BEGIN
select * into v_employee from employees where id=2;
dbms_output.put_line('l'identifiant de l'employee :' || v_employee.id);
dbms_output.put_line('le nom de l'employee :' || v_employee.name);
dbms_output.put_line('le departement de l'employee :' || v_employee.department);
dbms_output.put_line('le salaire l'employee :' || v_employee.salary);

END;
```

Sortie de script x

Tâche terminée en 0,152 secondes

l'identifiant de l'employee :2
le nom de l'employee :N2
le departement de l'employee :D2
le salaire l'employee :20

Procédure PL/SQL terminée.

Gestion des tableaux

Un tableau en PL/SQL est en fait une table qui ne comporte qu'une seule colonne. Cette colonne ne peut correspondre qu'à un type scalaire et nom à un type composé (table ou record)

La structure de données nécessaire pour gérer une table ORACLE nécessitera donc autant de types tables que la table possède de colonnes. On pourra éventuellement regrouper ces tables à l'intérieur d'un RECORD (Enregistrement)

0	1	2	3	4	5
Text0	Text1	Text2	Text3	Text4	Text5

```
TYPE tableau IS TABLE OF VARCHAR(20)  
INDEX BY BINARY_INTEGER;
```

Gestion des tableaux

```
DECLARE
    TYPE    nom_type_tableau    IS TABLE OF VARCHAR(20)
                                INDEX BY BINARY_INTEGER;
-- Déclaration de la variable tab de type nom_type_tableau

    tab    nom_type_tableau ;

BEGIN
    tab(1) := 'ligne 1' ;
    tab(2) := 'ligne 2' ;

    -- Affichage du premier élément de la variable tab,
    -- c'est à dire "ligne 1"
    IF tab.EXISTS(1) THEN
        DBMS_OUTPUT.PUT_LINE(tab(1)) ;
    END IF;

END ;
```

Gestion des tableaux

Les fonctions d'un tableau

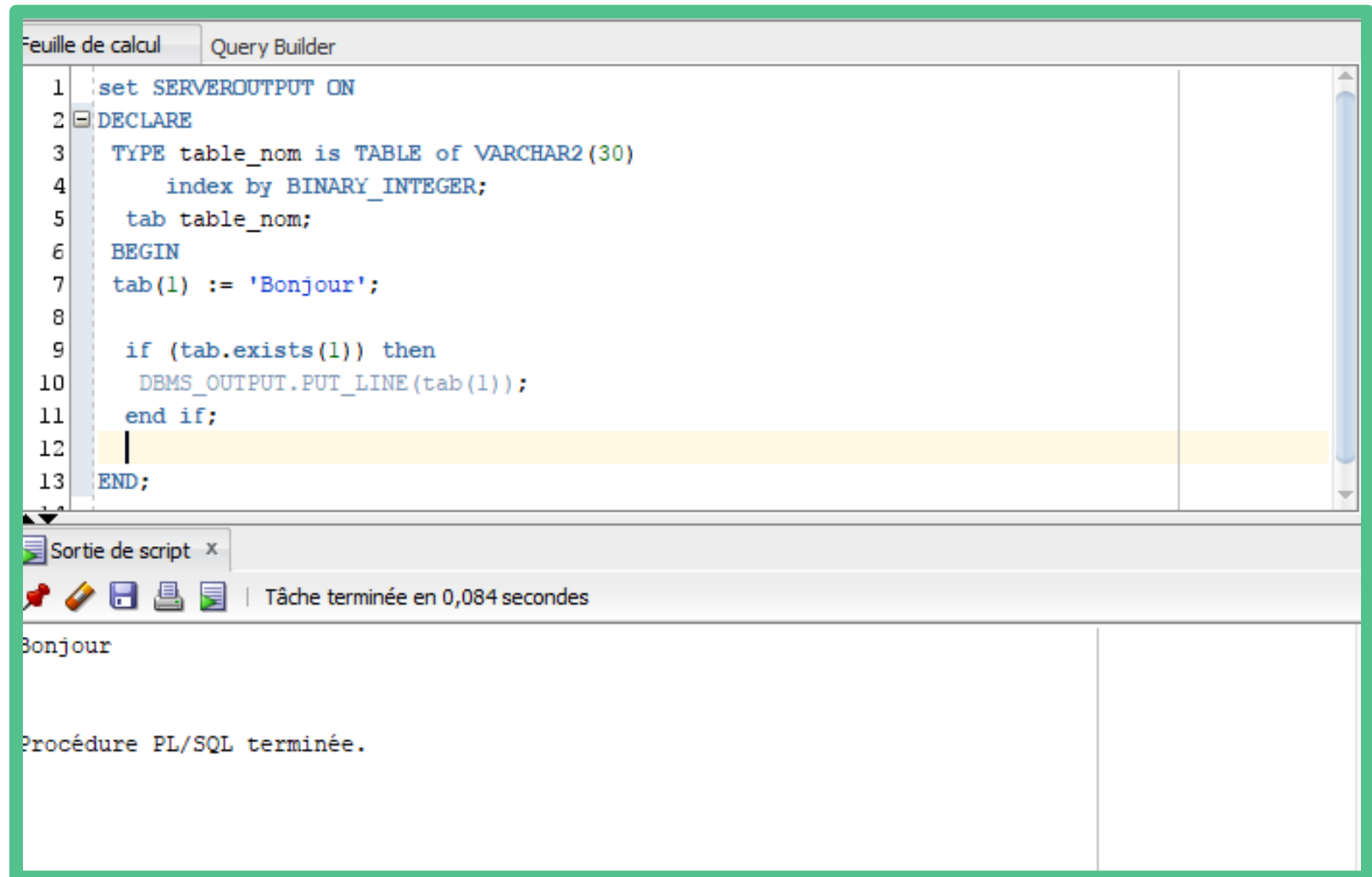
Fonction	
COUNT	retourne le nombre d'éléments de la table
EXISTS (n)	retourne TRUE si l'élément n existe
LAST	retourne la valeur du dernier index
FIRST	retourne le premier index
NEXT (n)	élément suivant
PRIOR(n)	élément précédent
DELETE	supprime toute la table
DELETE(n)	supprime l'élément n

Exemple :

```
IF table_name.EXISTS(1) THEN  
    INSERT INTO ...  
    ...
```

Gestion des tableaux

Exemple 1 :



The screenshot displays the Oracle SQL Query Builder interface. The main window is titled 'Feuille de calcul' and 'Query Builder'. It contains a PL/SQL script with the following code:

```
1 set SERVEROUTPUT ON
2 DECLARE
3     TYPE table_nom is TABLE of VARCHAR2(30)
4         index by BINARY_INTEGER;
5     tab table_nom;
6 BEGIN
7     tab(1) := 'Bonjour';
8
9     if (tab.exists(1)) then
10         DBMS_OUTPUT.PUT_LINE(tab(1));
11     end if;
12
13 END;
```

Below the script editor, there is a tab labeled 'Sortie de script'. The output shows the result of the script execution:

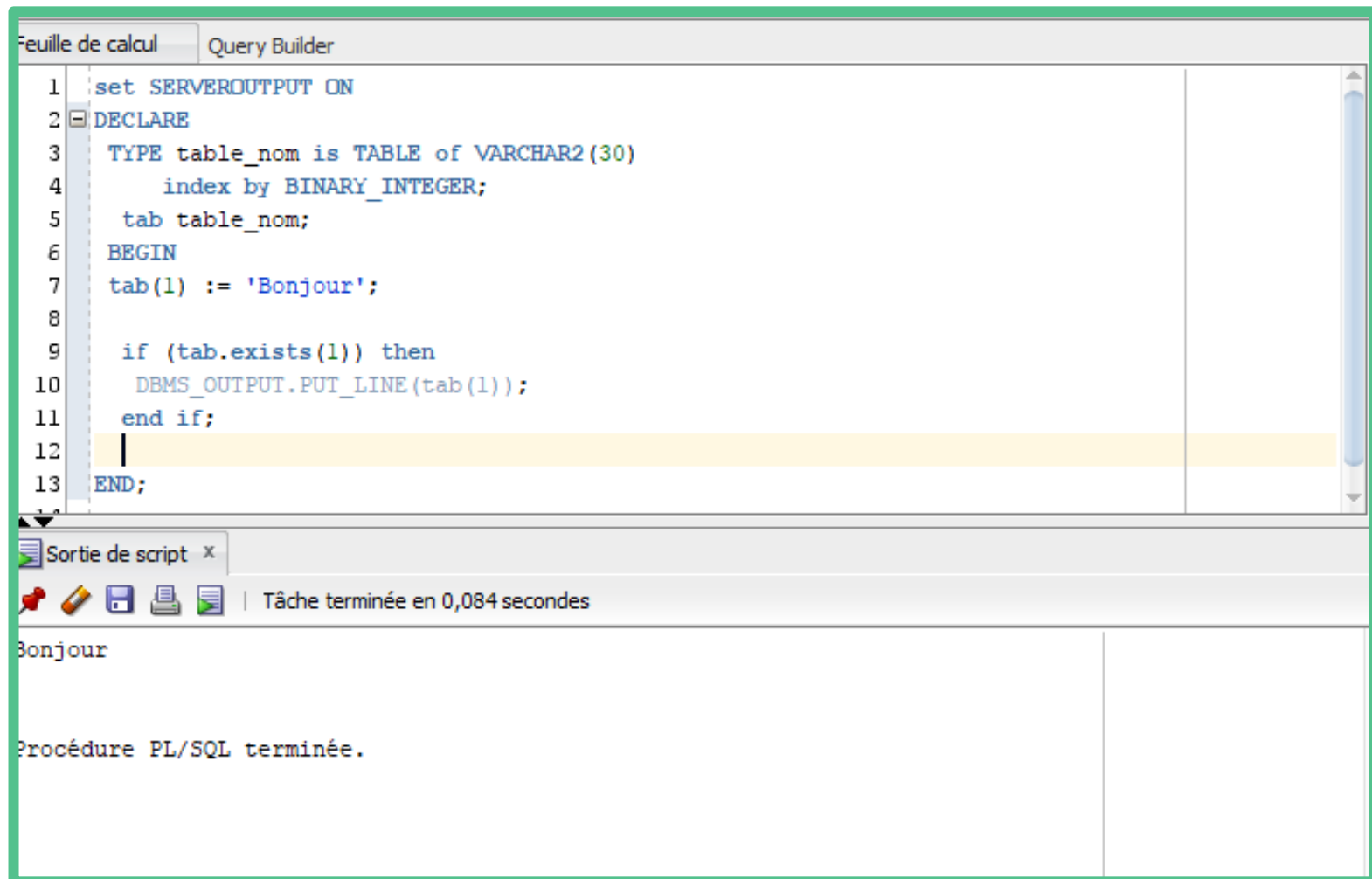
```
Bonjour

Procédure PL/SQL terminée.
```

The status bar at the bottom indicates 'Tâche terminée en 0,084 secondes'.

Gestion des tableaux

Exemple 2 :



The screenshot shows a software interface with two main panes. The top pane, titled 'Query Builder', contains a PL/SQL script. The script defines a table type 'table_nom' of 'VARCHAR2(30)' indexed by 'BINARY_INTEGER', declares a variable 'tab' of this type, and then checks if 'tab(1)' exists. If it does, it outputs the value 'Bonjour'. The bottom pane, titled 'Sortie de script', shows the output of the script, which is 'Bonjour' followed by a blank line and the message 'Procédure PL/SQL terminée.'.

```
1 set SERVEROUTPUT ON
2 DECLARE
3     TYPE table_nom is TABLE of VARCHAR2(30)
4         index by BINARY_INTEGER;
5     tab table_nom;
6 BEGIN
7     tab(1) := 'Bonjour';
8
9     if (tab.exists(1)) then
10         DBMS_OUTPUT.PUT_LINE(tab(1));
11     end if;
12
13 END;
```

Sortie de script x

Tâche terminée en 0,084 secondes

Bonjour

Procédure PL/SQL terminée.

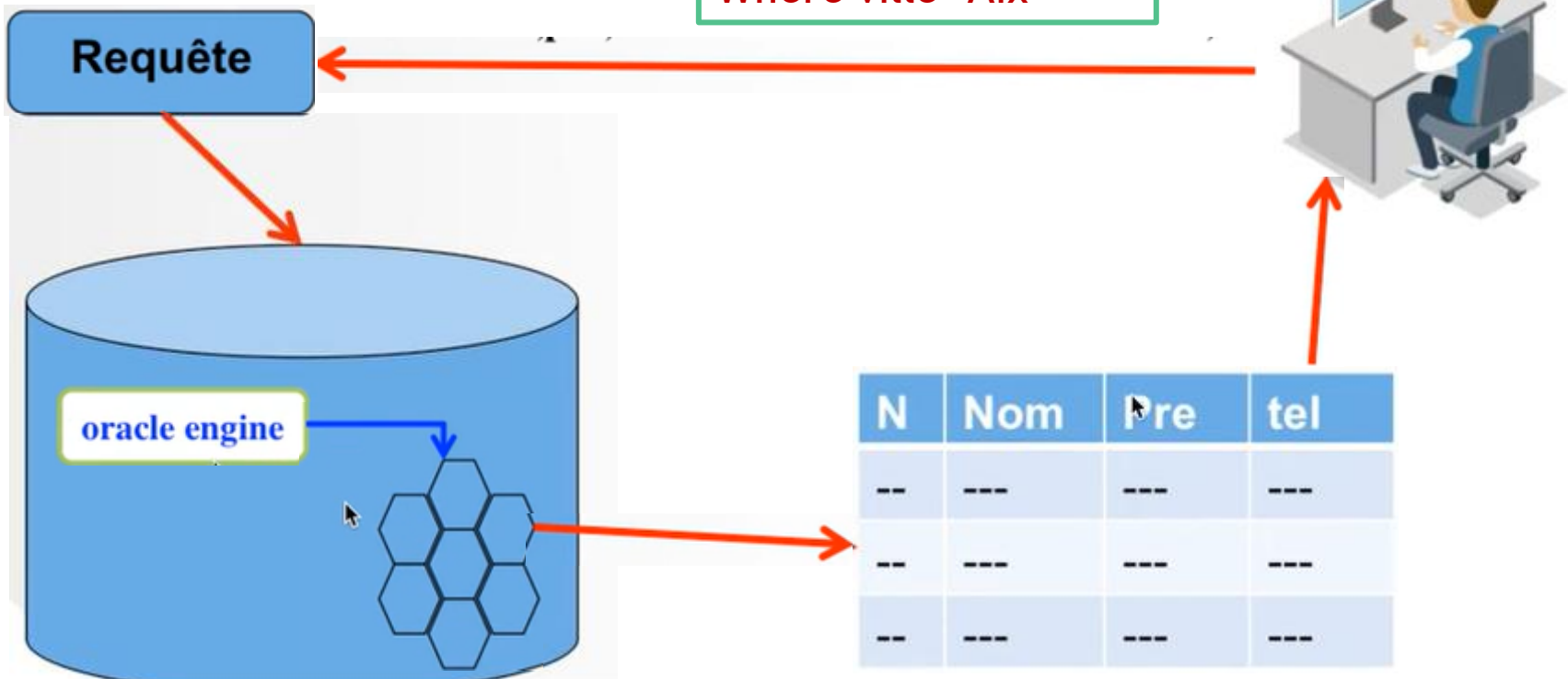
PL/SQL
Les curseurs

Contrôler les curseurs explicites

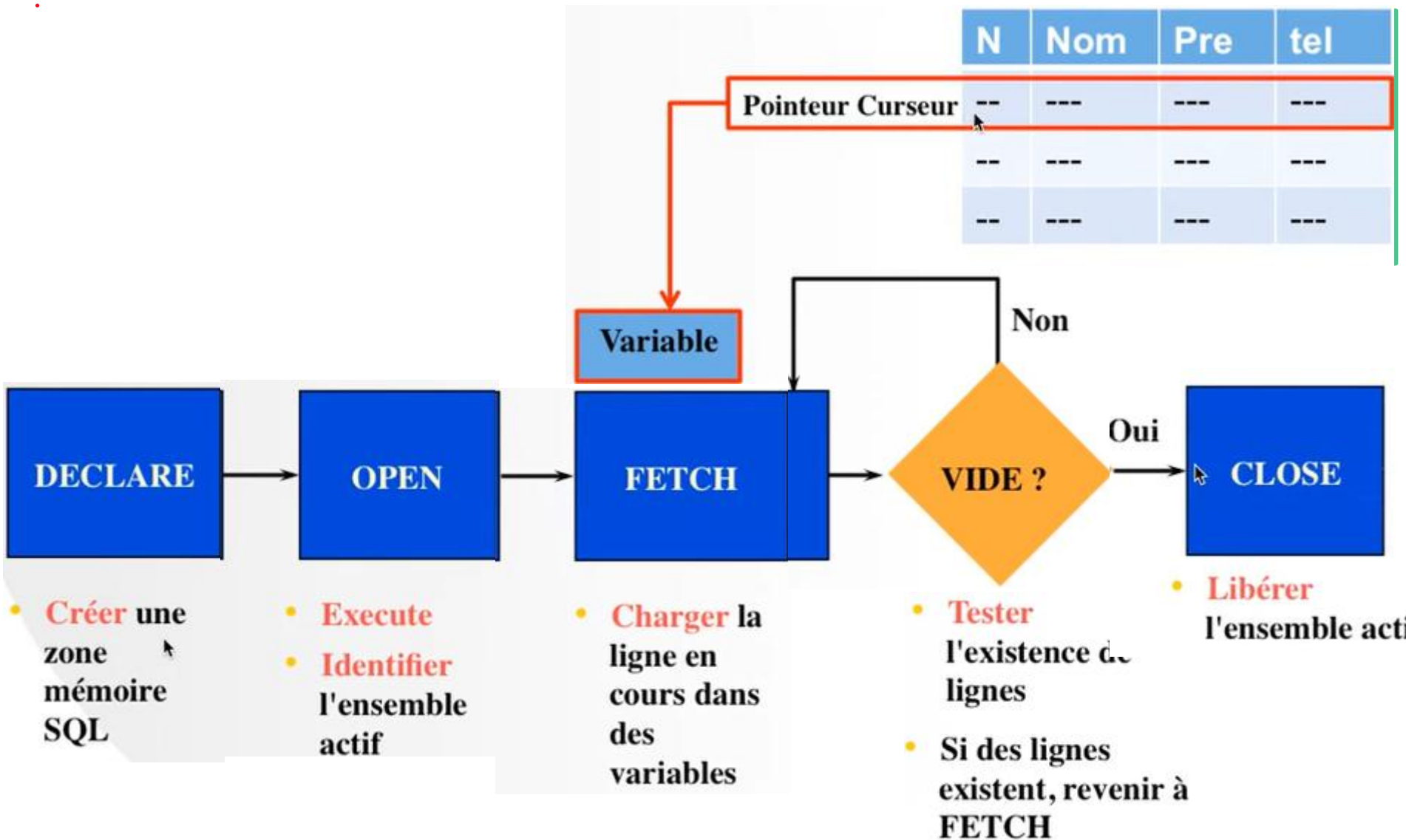
Les curseurs

Un **curseur** est une variable qui pointe vers le **résultat d'une requête SQL**. La déclaration du curseur est liée au texte de la requête.

```
Select nom, pre, tel  
From client  
Where ville='Aix'
```



Contrôler les curseurs explicites



Déclarer le curseur

Syntaxe:

DECLARE

```
CURSOR nom_curseur IS Requête_SQL
```

Exemple :

```
DECLARE
  CURSOR c1 IS
    SELECT employe_id, nom
    FROM   employe;
  CURSOR c2 IS
    SELECT *
    FROM   department
    WHERE  department_id = 10;
BEGIN
  ...
```

Ouvrir le curseur

Syntaxe:

```
OPEN nom_curseur;
```

- ☐ Ouvrir le curseur pour exécuter l'interrogation
- ☐ Si l'interrogation ne renvoie pas de ligne, aucune exception n'est déclenchée
- ☐ Utiliser les attributs du curseur pour tester le résultat après une extraction

Ouvrir le curseur

Syntaxe:

```
CLOSE nom_curseur;
```

- ❑ **Fermer** le curseur après avoir terminé le traitement des lignes
- ❑ **Rouvrir** le curseur, si nécessaire
- ❑ Ne pas essayer d'extraire les données d'un curseur s'il a été fermé

Extraire les données à partir du curseur

Syntaxe:

```
FETCH nom_curseur INTO var_curseur;
```

```
DECLARE
    v_empid  employe.employe_id%TYPE;
    v_nom    employe.nom%TYPE;
CURSOR c1 IS
    SELECT employe_id, nom
    FROM   employe;
BEGIN
    OPEN c1;
    FOR i IN 1..10 LOOP
        FETCH c1 INTO v_empid, v_nom;
        ...
    END LOOP;
    CLOSE c1;
END ;
```

Attributs d'un curseur explicite

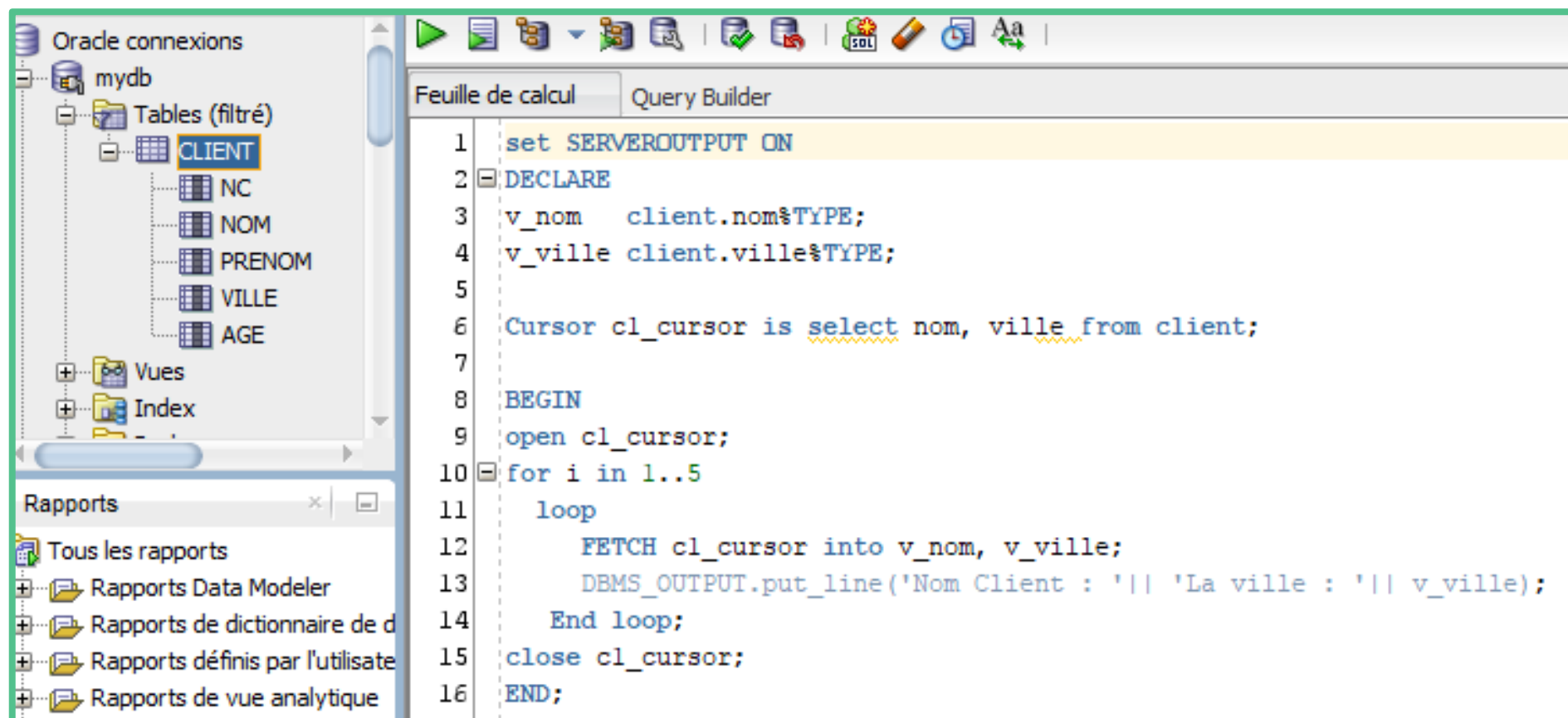
Obtenir les informations d'état concernant un curseur :

`nom_curseur%attribut`

Attribut	Type	Description
<code>%ISOPEN</code>	BOOLEAN	Prend la valeur TRUE si le curseur est ouvert
<code>%NOTFOUND</code>	BOOLEAN	Prend la valeur TRUE si la dernière extraction ne renvoie pas de ligne
<code>%FOUND</code>	BOOLEAN	Prend la valeur TRUE si la dernière extraction renvoie une ligne ; complément de <code>%NOTFOUND</code>
<code>%ROWCOUNT</code>	NUMBER	Prend la valeur correspondant au nombre total de lignes renvoyées jusqu'à présent

Attributs d'un curseur explicite

Boucle FOR pour un curseur



The screenshot displays the Oracle SQL Developer interface. On the left, the 'Orade connexions' tree shows a connection to 'mydb' with a table 'CLIENT' selected. The table's columns are listed: NC, NOM, PRENOM, VILLE, and AGE. Below this, the 'Rapports' section is visible. The main area, titled 'Feuille de calcul' and 'Query Builder', contains a PL/SQL script. The script sets 'SERVEROUTPUT ON', declares a cursor 'cl_cursor' for a query on 'CLIENT', and uses a 'FOR' loop to iterate through the results, printing each row's data.

```
1 set SERVEROUTPUT ON
2 DECLARE
3   v_nom   client.nom%TYPE;
4   v_ville client.ville%TYPE;
5
6   Cursor cl_cursor is select nom, ville from client;
7
8 BEGIN
9   open cl_cursor;
10  for i in 1..5
11    loop
12      FETCH cl_cursor into v_nom, v_ville;
13      DBMS_OUTPUT.put_line('Nom Client : '|| 'La ville : '|| v_ville);
14    End loop;
15  close cl_cursor;
16 END;
```

Attributs d'un curseur explicite

Boucle LOOP pour un curseur

```
1  set SERVEROUTPUT ON
2  DECLARE
3  v_nom    client.nom%TYPE;
4  v_ville  client.ville%TYPE;
5
6  Cursor cl_cursor is select nom, ville from client;
7
8  BEGIN
9  open cl_cursor;
10 Loop
11     FETCH cl_cursor into v_nom, v_ville;
12     exit when cl_cursor%notfound ;
13     DBMS_OUTPUT.put_line('Nom Client : '|| 'La ville : '|| v_ville);
14 End loop;
15 close cl_cursor;
16 END;
```



PL/SQL

Les exceptions

Gestion des erreurs

Le langage PL/SQL offre au développeur un mécanisme de gestion des exceptions.

Il permet de préciser la logique du traitement des erreurs survenues dans un bloc PL/SQL

Il s'agit donc d'un point clé dans l'efficacité du langage qui permettra de **protéger l'intégrité du système**.

- **Interne** : Les exceptions internes sont générées par le moteur du système (division par zéro, connexion non établie, table inexistante, privilèges insuffisants, mémoire saturée, espace disque insuffisant,...)
- **Externe** : Les exceptions externes sont générées par l'utilisateur.

Gestion des erreurs

Les erreurs ORACLE générées par le noyau **sont numérotées** (ORA-xxxxx)

Il a donc fallu établir une **table de correspondance** entre les erreurs ORACLE et des noms d'exceptions.

Voici quelques exemples **d'exceptions prédéfinis** et des codes correspondants:

Nom d'exception	Erreur ORACLE	SQLCODE
CURSOR_ALREADY_OPEN	ORA-06511	-6511
DUP_VAL_ON_INDEX	ORA-00001	-1
INVALID_CURSOR	ORA-01001	-1001
INVALID_NUMBER	ORA-01722	-1722
LOGIN_DENIED	ORA-01017	-1017
NO_DATA_FOUND	ORA-01403	+100
PROGRAM_ERROR	ORA-06501	-6501
ROWTYPE_MISMATCH	ORA-06504	-6504
TIMEOUT_ON_RESOURCE	ORA-00051	-51
TOO_MANY_ROWS	ORA-01422	-1422
ZERO_DIVIDE	ORA-01476	-1476

Gestion des erreurs

Signification des erreurs Oracles présentées ci-dessus :

- **CURSOR_ALREADY_OPEN** : tentative d'ouverture d'un curseur déjà ouvert.
- **DUP_VAL_ON_INDEX** : violation de l'unicité lors d'une mise à jour détectée au niveau de l'index unique
- **INVALID_CURSOR**: opération incorrecte sur un curseur, comme par exemple la fermeture d'un curseur qui n'a pas été ouvert.
- **INVALID_NUMBER**: échec de la conversion d'une chaîne de caractères en numérique
- **LOGIN_DENIED**: connexion à la base échouée car le nom utilisateur ou le mot de passe est invalide.
- **NO_DATA_FOUND**: déclenché si la commande SELECT INTO ne retourne aucune ligne ou si on fait référence à un enregistrement non initialisé d'un tableau PL/SQL
- **PROGRAM_ERROR**: problème général du au PL/SQL.
- **ROWTYPE_MISMATCH**: survient lorsque une variable curseur d'un programme hôte retourne une valeur dans une variable curseur d'un bloc PL/SQL qui n'a pas le même type.
- **TIMEOUT_ON_RESOURCE**: dépassement du temps dans l'attente de libération des ressources (lié aux paramètres de la table).
- **TOO_MANY_ROWS**: la commande SELECT INTO retourne plus d'une ligne.
- **ZERO_DIVIDE**: tentative de division par zéro

Exception utilisateur

Syntaxe:

[DECLARE]

nom_erreur EXCEPTION;

1

BEGIN

IF (anomalie) THEN
RAISE nom_erreur;
END IF;

2

[EXCEPTION]

WHEN nom_erreur THEN traitement;

3

END;

Exception utilisateur

Exemple : Exception utilisateur

```
DECLARE
  Employe_age INT;
  invalid_age EXCEPTION;
BEGIN
  SELECT age INTO Employe_age FROM Employe WHERE nom='Salami';

  IF Employe_age <= 0 THEN
    RAISE invalid_age;
  ELSE
    DBMS_OUTPUT.PUT_LINE('L age de l'employé est: '||Employe_age);
  END IF;

  EXCEPTION
  WHEN invalid_age THEN dbms_output.put_line('L age est invalide');

END;
```

Exception Oracle - Prédéfinie

Syntaxe

```
BEGIN  SELECT ... COMMIT;
EXCEPTION
  WHEN NO_DATA_FOUND THEN
    statement1;
    statement2;
  WHEN TOO_MANY_ROWS THEN
    statement1;
  WHEN OTHERS THEN
    statement1;
    statement2;
    statement3;
END;
```

Exception Oracle – Prédéfinie

Exemple : Exception prédéfinie Oracle

```
DECLARE
Var_Nom EMPLOYEE.nom%TYPE;
BEGIN
SELECT nom INTO Var_Nom FROM EMPLOYEE WHERE numéro=1;

EXCEPTION
WHEN NO_DATA_FOUND THEN
    DBMS_OUTPUT.PUT_LINE('L'employé n'existe pas') ;

WHEN OTHERS THEN
    DBMS_OUTPUT.PUT_LINE('Erreur' ) ;
END;
/
```

Exception Oracle – Non Prédéfinie

Dans ce type nous créons une **correspondance** entre le code erreur ORACLE et le nom de l'**exception**.

Ce nom est choisi librement par le programmeur.

Pour affecter un nom à un code erreur, on doit utiliser une directive compilée (pragma) nommée **EXCEPTION_INIT**.

Syntaxe

[DECLARE]

```
champ_obligatoire EXCEPTION;
```

1

```
PRAGMA EXCEPTION_INIT (champ_obligatoire, -1400);
```

2

BEGIN...

```
EXCEPTION
```

```
WHEN champ_obligatoire THEN traitement;
```

3

END;

Exception Oracle – Non Prédéfinie

Exemple : Exception Orale – non prédéfinie

```
DECLARE  
Erreur_data EXCEPTION;  
Var_Nom EMPLOYE.nom%TYPE;  
PRAGMA EXCEPTION_INIT (Erreur_data , -100);  
BEGIN  
SELECT nom INTO var_nom FROM Employé WHERE code=1;  
EXCEPTION  
    WHEN Erreur_data THEN  
    DBMS_OUTPUT.PUT_LINE('L'employé n'existe pas' );  
END;
```

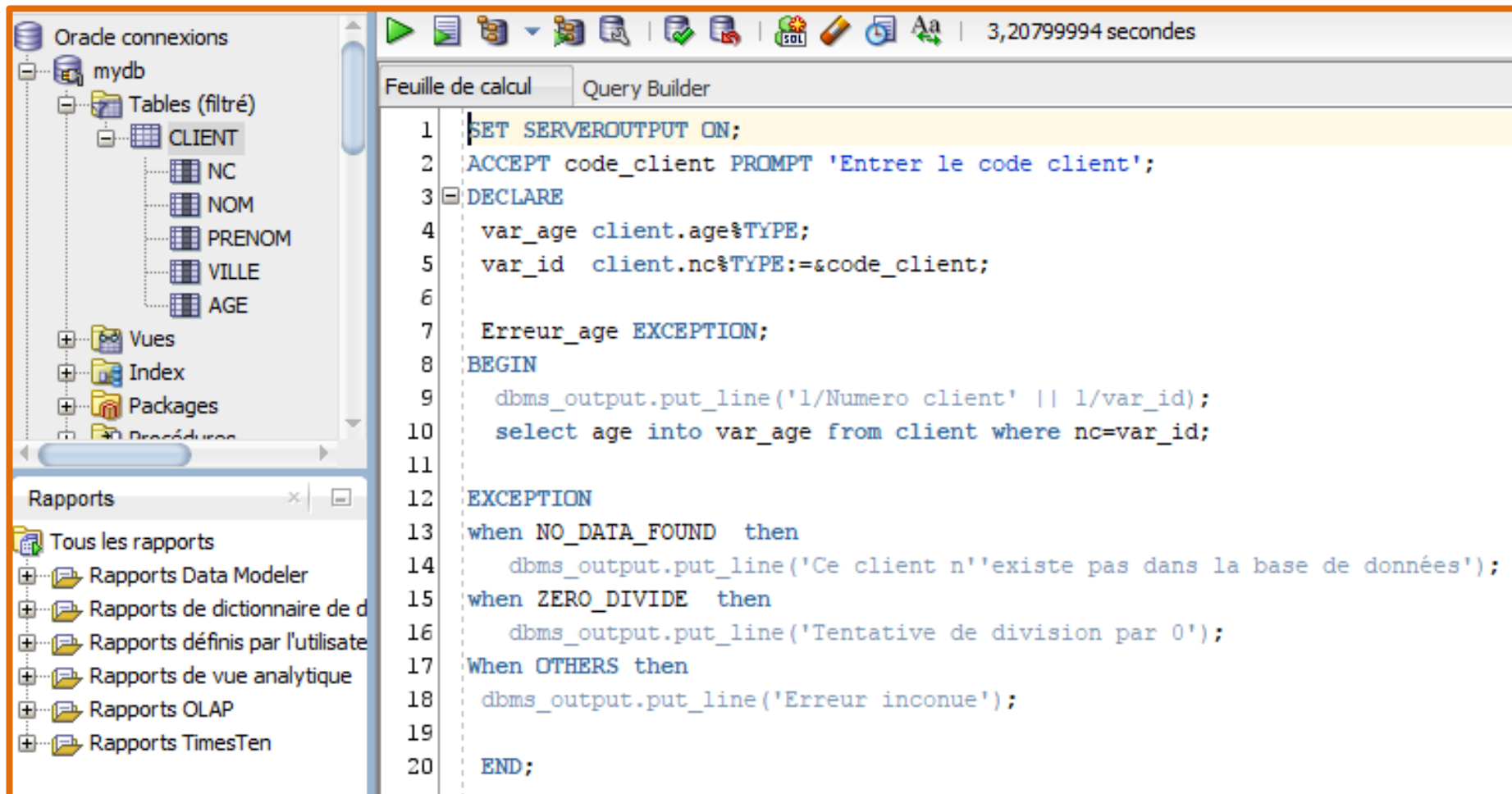
Le code erreur -100 correspond aux données inexistantes .

Exemple : Exception utilisateur

The screenshot displays the Oracle SQL Developer interface. On the left, the 'Orade connexions' tree shows a connection to 'mydb' with a filtered table view of 'CLIENT' containing columns NC, NOM, PRENOM, VILLE, and AGE. Below this, the 'Rapports' section lists various report types. The main window is the 'Query Builder' tab, showing a PL/SQL script with the following code:

```
1 SET SERVEROUTPUT ON;
2 ACCEPT code_client PROMPT 'Entrer le code client';
3 DECLARE
4   var_age client.age%TYPE;
5   var_id  client.nc%TYPE:=&code_client;
6
7   Erreur_age EXCEPTION;
8 BEGIN
9   select age into var_age from client where nc=var_id;
10
11   if (var_age<20) then
12
13     raise Erreur_age;
14   else
15     dbms_output.put_line('Traitement normale');
16   end if;
17
18 EXCEPTION
19 when Erreur_age then
20   update client set age=20 where nc=var_id;
21 END;
```

Exception Oracle - Prédéfinie



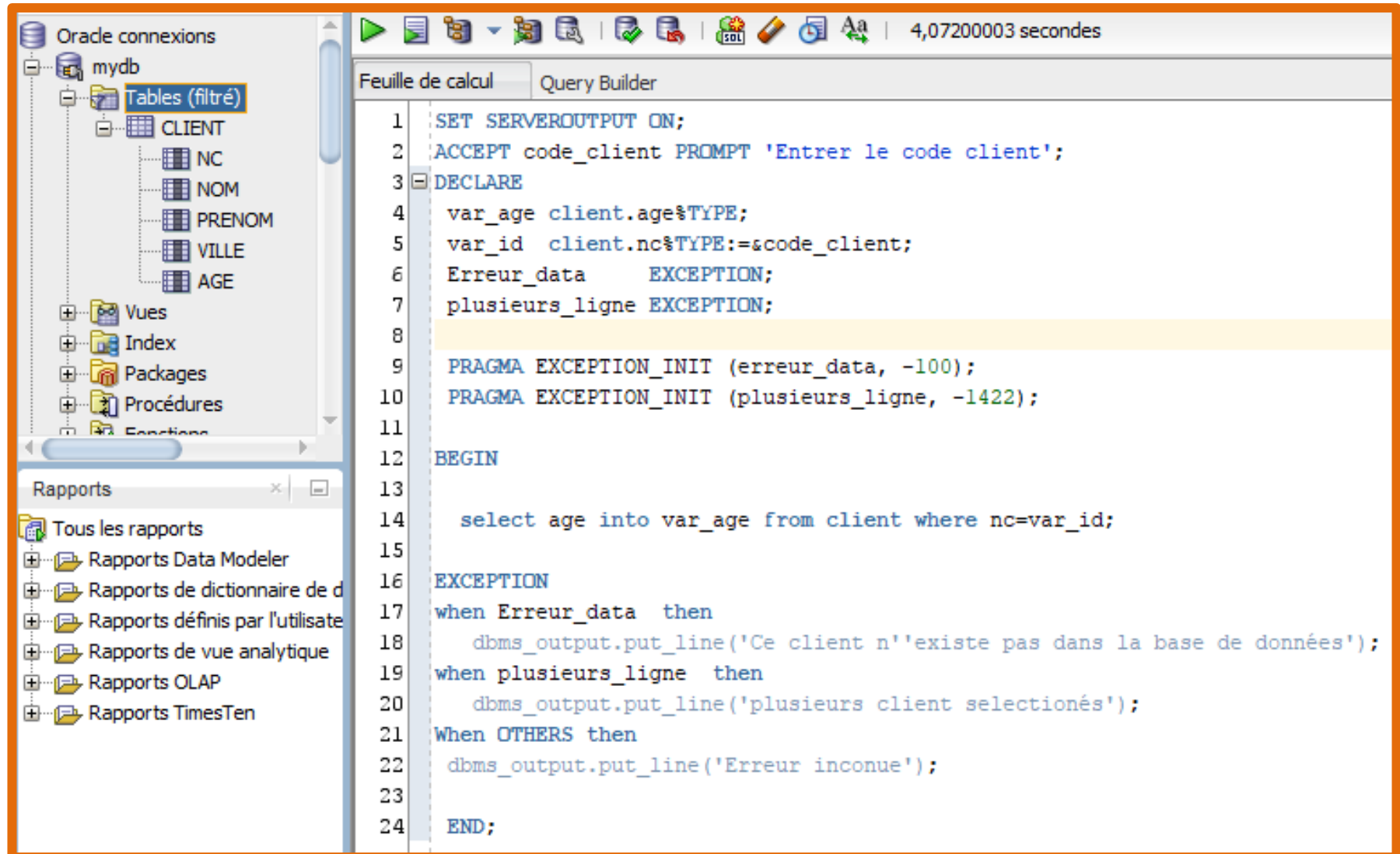
The screenshot displays the Oracle SQL Developer environment. On the left, the 'Oracle connexions' tree shows a connection to 'mydb' with a filtered view of the 'CLIENT' table, listing columns: NC, NOM, PRENOM, VILLE, and AGE. Below this, the 'Rapports' (Reports) section is visible, listing various report types like 'Rapports Data Modeler', 'Rapports de dictionnaire de données', etc.

The main workspace is titled 'Feuille de calcul' (Spreadsheet) and 'Query Builder'. It contains a PL/SQL script with the following code:

```
1 SET SERVEROUTPUT ON;
2 ACCEPT code_client PROMPT 'Entrer le code client';
3 DECLARE
4   var_age client.age%TYPE;
5   var_id  client.nc%TYPE:=&code_client;
6
7   Erreur_age EXCEPTION;
8 BEGIN
9   dbms_output.put_line('1/Numero client' || 1/var_id);
10  select age into var_age from client where nc=var_id;
11
12 EXCEPTION
13 when NO_DATA_FOUND then
14   dbms_output.put_line('Ce client n'existe pas dans la base de données');
15 when ZERO_DIVIDE then
16   dbms_output.put_line('Tentative de division par 0');
17 When OTHERS then
18   dbms_output.put_line('Erreur inconnue');
19
20 END;
```

The script is designed to prompt the user for a client code, attempt a query, and handle specific Oracle exceptions: `NO_DATA_FOUND` (client not found) and `ZERO_DIVIDE` (division by zero), as well as any other unexpected errors.

Exception Oracle - Non Prédéfinie



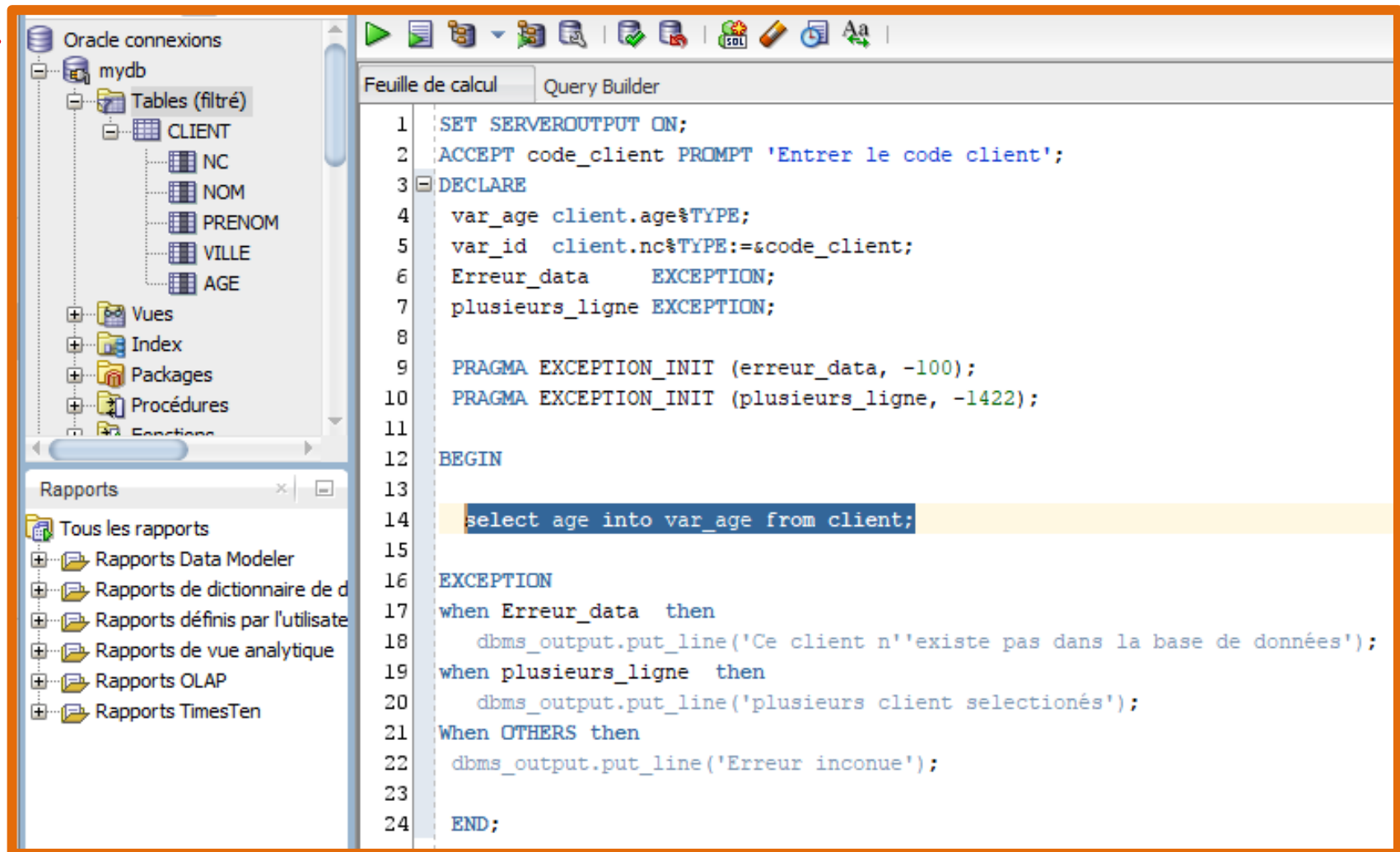
The screenshot displays the Oracle SQL Developer interface. On the left, the 'Oracle connexions' tree shows a connection to 'mydb'. Under 'Tables (filtré)', the 'CLIENT' table is expanded, showing columns: NC, NOM, PRENOM, VILLE, and AGE. Below this, the 'Rapports' section lists various report types, including 'Rapports Data Modeler', 'Rapports de dictionnaire de données', 'Rapports définis par l'utilisateur', 'Rapports de vue analytique', 'Rapports OLAP', and 'Rapports TimesTen'.

The main window is titled 'Feuille de calcul' and 'Query Builder'. It shows a PL/SQL script with the following code:

```
1 SET SERVEROUTPUT ON;
2 ACCEPT code_client PROMPT 'Entrer le code client';
3 DECLARE
4   var_age client.age%TYPE;
5   var_id  client.nc%TYPE:=&code_client;
6   Erreur_data      EXCEPTION;
7   plusieurs_ligne  EXCEPTION;
8
9   PRAGMA EXCEPTION_INIT (erreur_data, -100);
10  PRAGMA EXCEPTION_INIT (plusieurs_ligne, -1422);
11
12 BEGIN
13
14   select age into var_age from client where nc=var_id;
15
16 EXCEPTION
17 when Erreur_data then
18   dbms_output.put_line('Ce client n'existe pas dans la base de données');
19 when plusieurs_ligne then
20   dbms_output.put_line('plusieurs client sélectionnés');
21 When OTHERS then
22   dbms_output.put_line('Erreur inconnue');
23
24 END;
```

The script is executed, and the execution time is displayed as 4,07200003 secondes.

Exception Oracle - Non Prédéfinie



The screenshot displays the Oracle SQL Developer interface. On the left, the 'Oracle connexions' tree shows a connection to 'mydb' with a schema containing tables like CLIENT, NC, NOM, PRENOM, VILLE, and AGE. The main window is titled 'Feuille de calcul' and 'Query Builder', showing a PL/SQL script with custom exceptions.

```
1 SET SERVEROUTPUT ON;
2 ACCEPT code_client PROMPT 'Entrer le code client';
3 DECLARE
4   var_age client.age%TYPE;
5   var_id  client.nc%TYPE:=&code_client;
6   Erreur_data      EXCEPTION;
7   plusieurs_ligne  EXCEPTION;
8
9   PRAGMA EXCEPTION_INIT (erreur_data, -100);
10  PRAGMA EXCEPTION_INIT (plusieurs_ligne, -1422);
11
12 BEGIN
13
14   select age into var_age from client;
15
16 EXCEPTION
17 when Erreur_data then
18   dbms_output.put_line('Ce client n''existe pas dans la base de données');
19 when plusieurs_ligne then
20   dbms_output.put_line('plusieurs client selectionés');
21 When OTHERS then
22   dbms_output.put_line('Erreur inconnue');
23
24 END;
```

Attributs d'un curseur explicite

```
Feuille de calcul | Query Builder
ACCEPT Code_Client PROMPT 'Entrez le Code Client : '
DECLARE
var_age client.age%type;
var_id client.nc%type:=&code_client;

BEGIN
DBMS_OUTPUT.PUT('1/Numero client'||1/var_id);
select age into var_age from client where nc=var_id;
insert into client values (1,1aaa,'aaa','aaa',12);

EXCEPTION
when NO_DATA_FOUND THEN
    DBMS_OUTPUT.PUT_LINE('Ce client n''existe pas dans la base de données');
when ZERO_DIVIDE then
    DBMS_OUTPUT.PUT_LINE('Tentative de division par 0');
when OTHERS then
    DBMS_OUTPUT.PUT_line('Erreur inconnue');

END;
```

Sortie de script x

Tâche terminée en 9,484 secondes

1/Numero client1Erreur inconnue

Procédure PL/SQL terminée.

curseur explicite

```
Feuille de calcul | Query Builder

SET SERVEROUTPUT ON;

ACCEPT code_client PROMPT 'Entrer le code client';

DECLARE
var_age client.age%type;
var_id client.nc%type:=&code_client;
Erreur_data EXCEPTION;
plusieurs_ligne EXCEPTION;

PRAGMA EXCEPTION_INIT (erreur_data, -4910);

BEGIN
DBMS_OUTPUT.PUT('1/Numero client'||1/var_id);
select age into var_age from client where nc=var_id;
insert into client values (1,'aaa','aaa','aaa',12);

EXCEPTION
when NO_DATA_FOUND THEN
    DBMS_OUTPUT.PUT_LINE('Ce client n''existe pas dans la base de données');
```

```
Feuille de calcul | Query Builder

SET SERVEROUTPUT ON;

ACCEPT code_client PROMPT 'Entrer le code client';

DECLARE
var_age client.age%type;
var_id client.nc%type:=&code_client;
Erreur_data EXCEPTION;
plusieurs_ligne EXCEPTION;

PRAGMA EXCEPTION_INIT (erreur_data, -100);
PRAGMA EXCEPTION_INIT (plusieurs_ligne, -1422);

BEGIN

select age into var_age from client where nc=var_id;
insert into client values (1,'aaa','aaa','aaa',12);

EXCEPTION
```


Attributs d'un curseur explicite

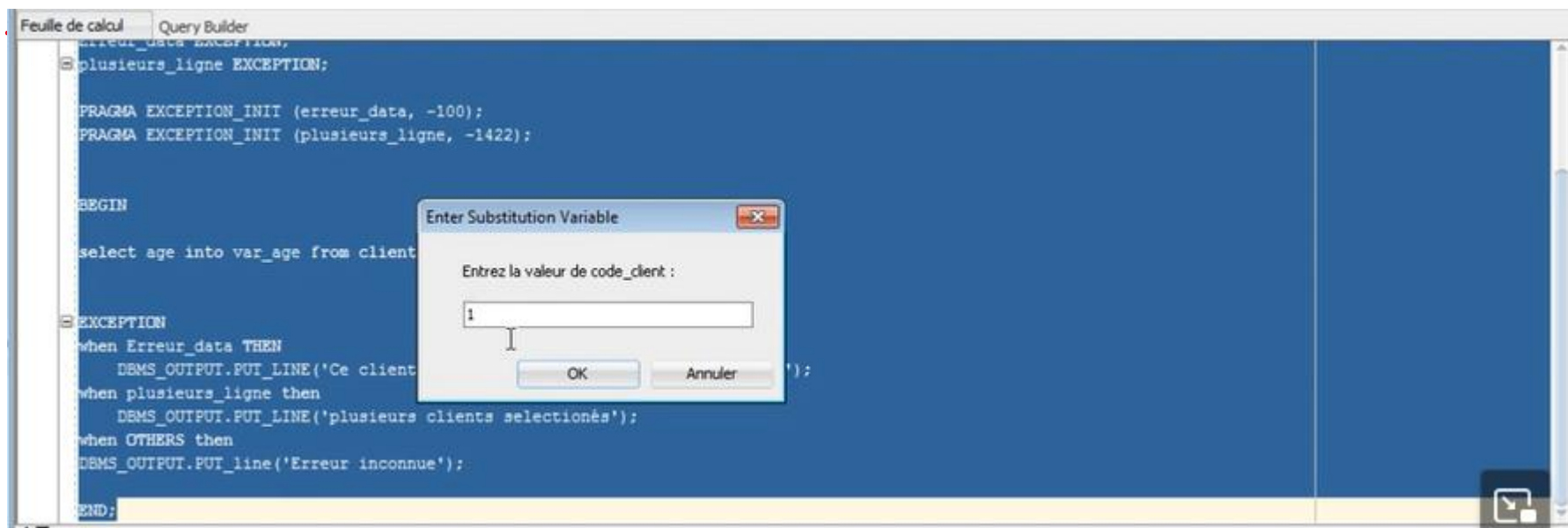


The screenshot shows a SQL Query Builder window with a tab labeled 'Feuille de calcul' and 'Query Builder'. The main area contains the following SQL code:

```
Erreur_data EXCEPTION;  
plusieurs_ligne EXCEPTION;  
  
PRAGMA EXCEPTION_INIT (erreur_data, -100);  
PRAGMA EXCEPTION_INIT (plusieurs_ligne, -1422);  
  
BEGIN  
  
select age into var_age from client where nc=var_id;  
  
EXCEPTION  
when Erreur_data THEN  
    DBMS_OUTPUT.PUT_LINE('Ce client n'existe pas dans la base de données');  
when plusieurs_ligne then  
    DBMS_OUTPUT.PUT_LINE('plusieurs clients selectionés');  
when OTHERS then  
    DBMS_OUTPUT.PUT_line('Erreur inconnue');  
  
END;
```

The code defines two exceptions, `Erreur_data` and `plusieurs_ligne`, and initializes them with specific error codes using `PRAGMA EXCEPTION_INIT`. The main block starts with `BEGIN`, followed by a `select` statement that attempts to insert data into `var_age` from the `client` table. The `EXCEPTION` section handles the errors: `when Erreur_data THEN` outputs 'Ce client n'existe pas dans la base de données', `when plusieurs_ligne then` outputs 'plusieurs clients selectionés', and `when OTHERS then` outputs 'Erreur inconnue'. The block ends with `END;`.

Attributs d'un curseur explicite

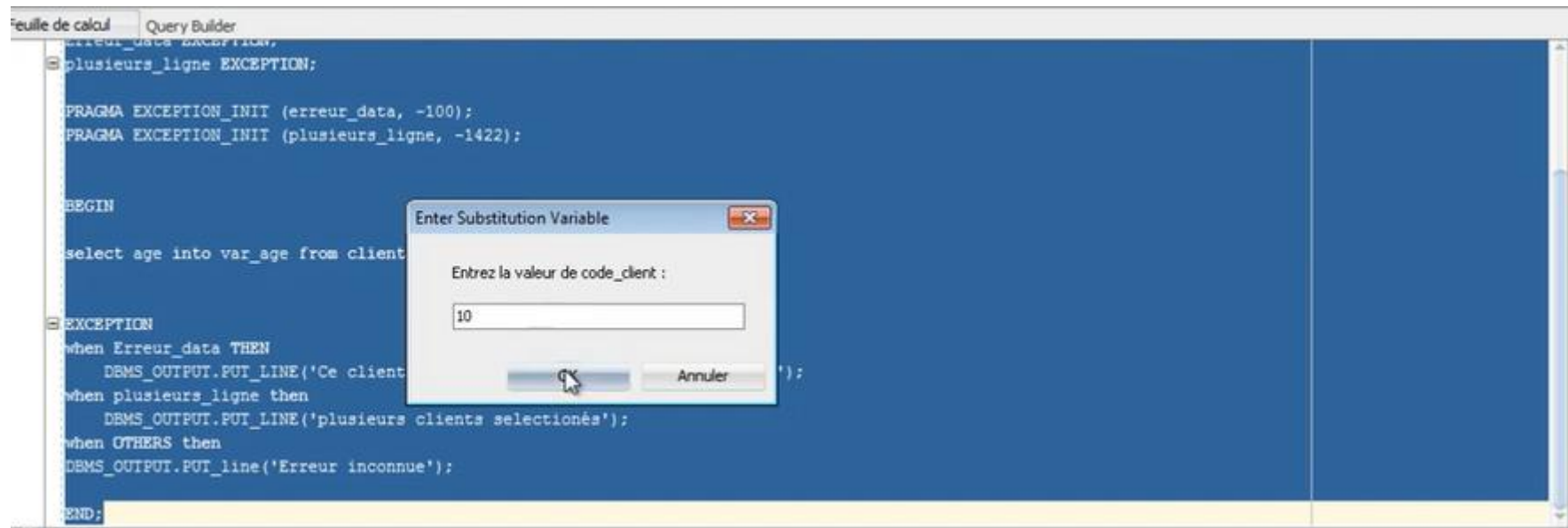


```
DBMS_OUTPUT.PUT_line('Erreur inconnue');
```

```
END;
```

```
Procédure PL/SQL terminée.
```

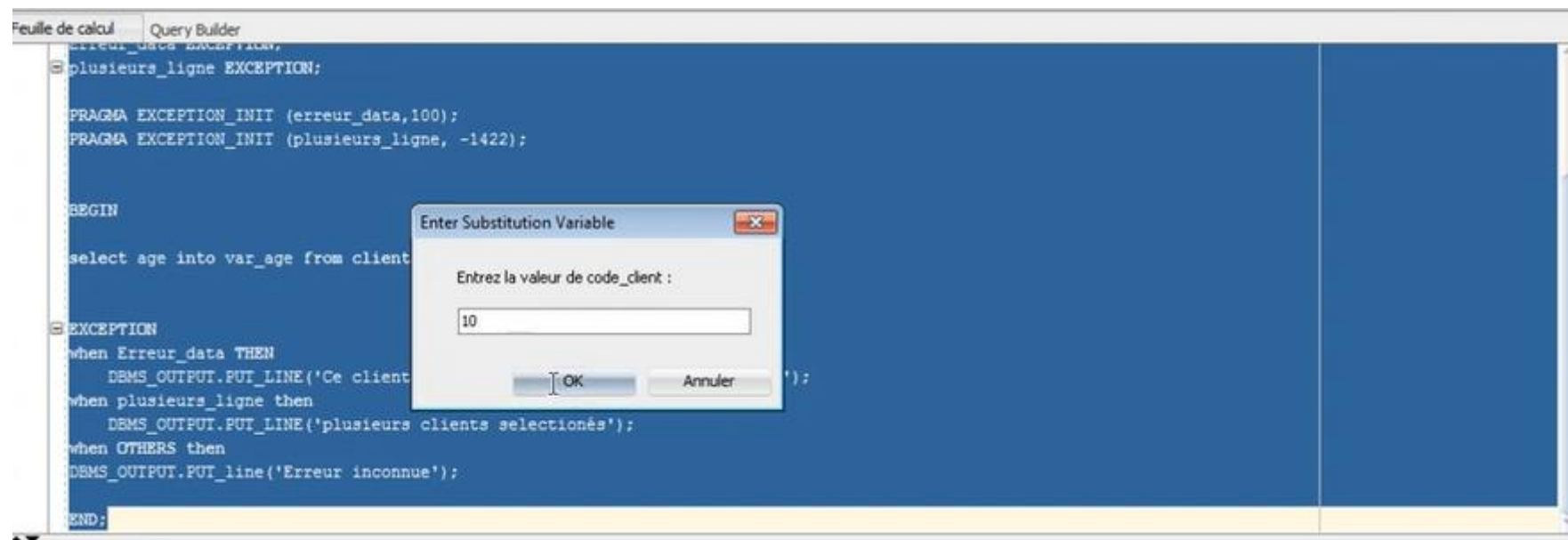
Attributs d'un curseur explicite



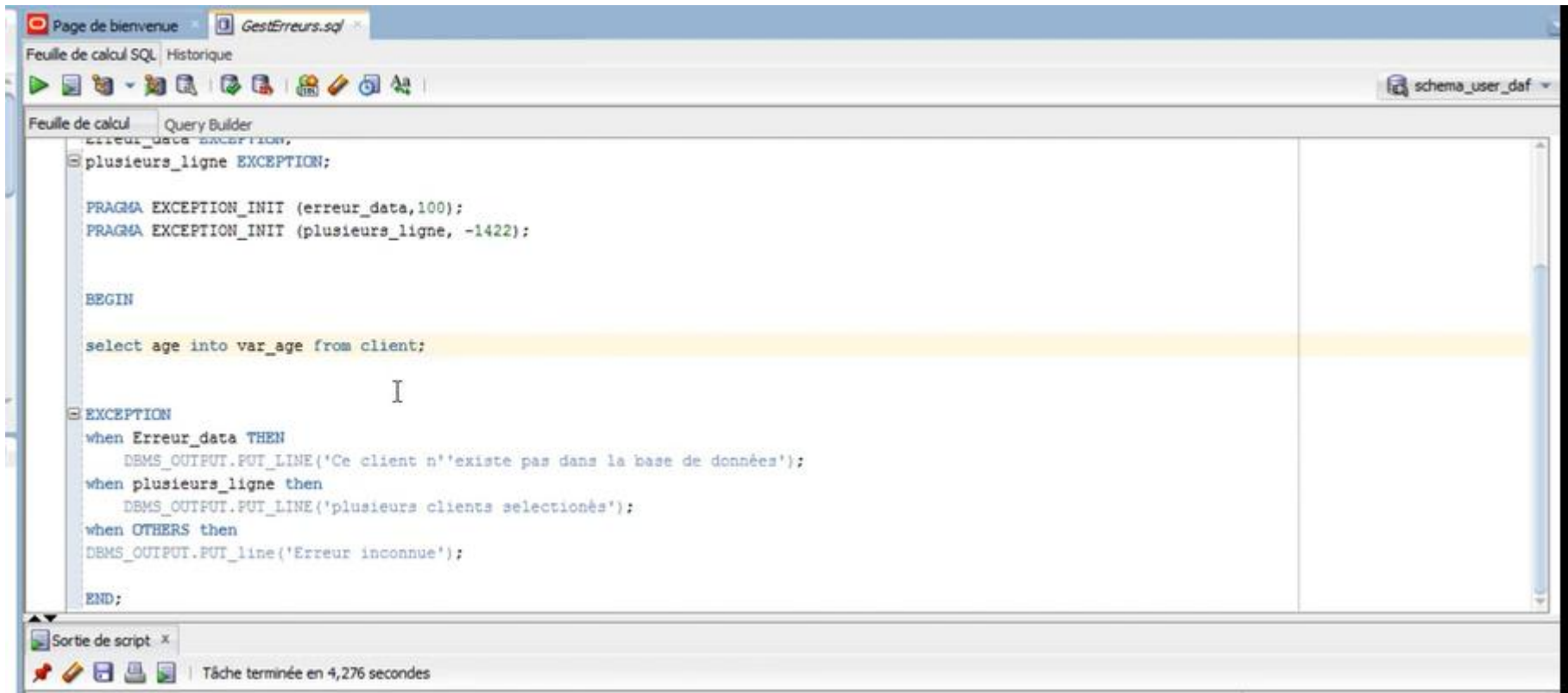
```
END;
Erreur inconnue

Procédure PL/SQL terminée.
```

Attributs d'un curseur explicite



Attributs d'un curseur explicite



The screenshot shows a SQL IDE window titled "GestErreurs.sql". The main editor displays a PL/SQL script. The script defines two cursor attributes: `erreur_data EXCEPTION;` and `plusieurs_ligne EXCEPTION;`. It then initializes these attributes using `PRAGMA EXCEPTION_INIT (erreur_data, 100);` and `PRAGMA EXCEPTION_INIT (plusieurs_ligne, -1422);`. The main block starts with `BEGIN` followed by `select age into var_age from client;`, which is highlighted in yellow. An exception block follows, starting with `EXCEPTION` and containing three `when` clauses: `when Erreur_data THEN`, `when plusieurs_ligne then`, and `when OTHERS then`. Each clause calls `DBMS_OUTPUT.PUT_LINE` with a specific message. The script ends with `END;`. The status bar at the bottom indicates "Tâche terminée en 4,276 secondes".

```
Page de bienvenue - GestErreurs.sql
Feuille de calcul SQL Historique
Feuille de calcul Query Builder
schema_user_def

-- Erreur_data EXCEPTION;
-- plusieurs_ligne EXCEPTION;

PRAGMA EXCEPTION_INIT (erreur_data, 100);
PRAGMA EXCEPTION_INIT (plusieurs_ligne, -1422);

BEGIN

select age into var_age from client;

EXCEPTION
when Erreur_data THEN
    DBMS_OUTPUT.PUT_LINE('Ce client n''existe pas dans la base de données');
when plusieurs_ligne then
    DBMS_OUTPUT.PUT_LINE('plusieurs clients selectionnés');
when OTHERS then
    DBMS_OUTPUT.PUT_line('Erreur inconnue');

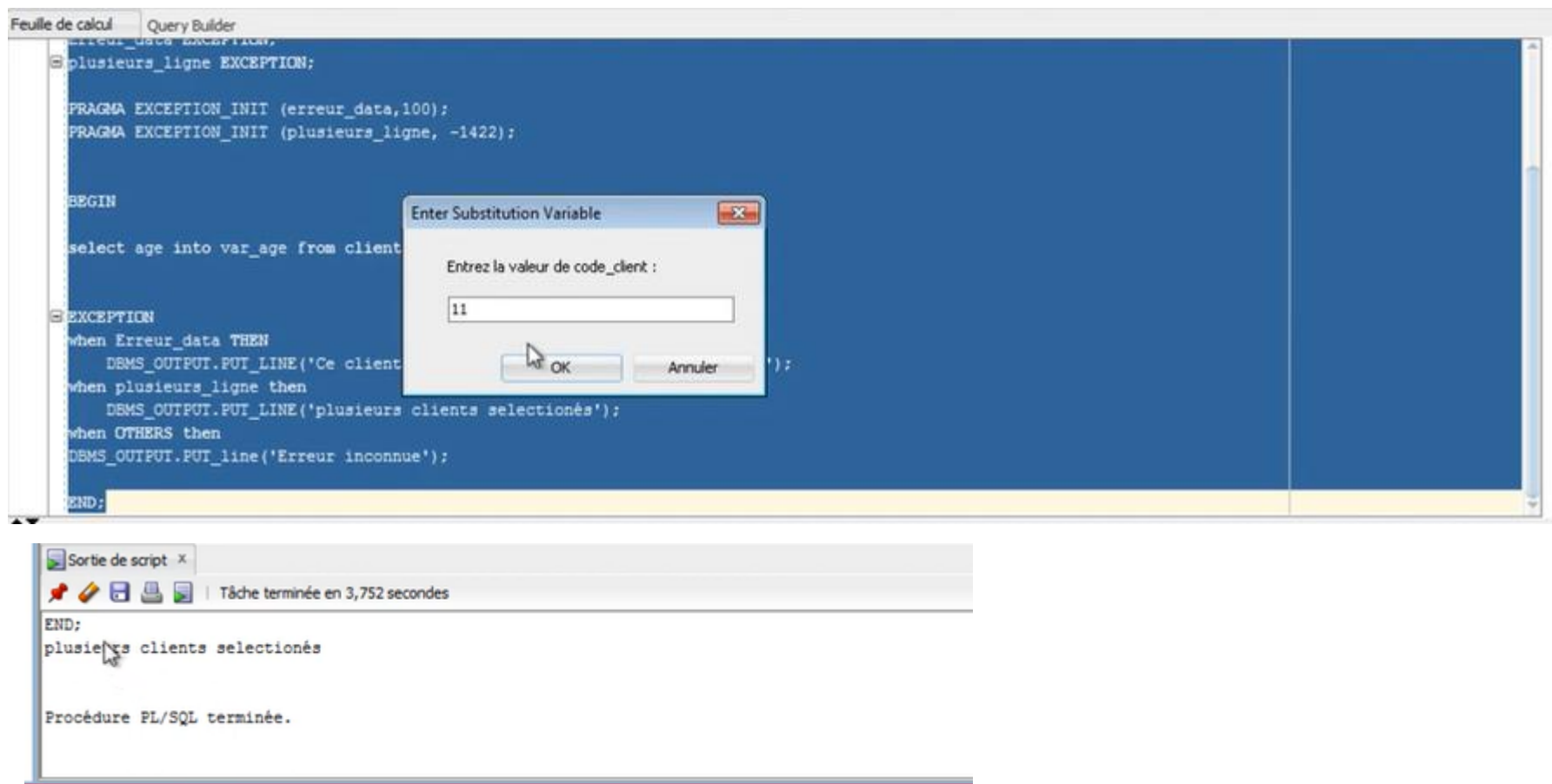
END;
```

Sortie de script x

Tâche terminée en 4,276 secondes

Attributs d'un curseur explicite

-
-



The image displays a PL/SQL Query Builder window with a procedure named `plusieurs_ligne EXCEPTION;`. The procedure includes `PRAGMA EXCEPTION_INIT` for `erreur_data` (100) and `plusieurs_ligne` (-1422). It begins with `select age into var_age from client`. An `EXCEPTION` block handles `Erreur_data` (outputting 'Ce client'), `plusieurs_ligne` (outputting 'plusieurs clients selectionés'), and `OTHERS` (outputting 'Erreur inconnue').

An "Enter Substitution Variable" dialog box is overlaid, asking for the value of `code_client`. The input field contains the value `11`, and the `OK` button is highlighted.

Below the Query Builder, a "Sortie de script" window shows the execution output:

```
END;
plusieurs clients selectionés

Procédure PL/SQL terminée.
```